Ecole Africaine de la Météorologie et de l'Aviation Civile
(EAMAC)

Division Électronique Informatique

PROJET DE FIN D'ETUDE

# Mise en œuvre d'un dispositif de calage automatique de l'angle d'inclinaison d'un plan

**EAC-SEI 2023-A**

**Rédigé et soutenu par :**

- ☞ AVOCEGAMOU Lionel Noukpo
- ☞ RABARY Haja Tahina

**Encadrant :**

DAKO Komlan Dominique Tavio

## Membre du Jury

- ☞ Nom du jury 1
- ☞ Nom du jury 2

11 juillet 2025

# *Dédicace*

Je dédie ce projet de fin d'études :

- À mes parents biologiques et à mon parrain, pour leur soutien indéfectible et leurs bénédictions constantes, qui m'ont accompagné tout au long de ce parcours.
- À l'ensemble de la promotion EAC SEI 2023 A - B, et plus particulièrement à mon binôme **RABARY Haja Tahina**, dont l'engagement et la collaboration ont été déterminants pour la réussite de ce travail.

AVOCEGAMOU Lionel N.

---

Je dédie ce projet à ma famille, en particulier à mes parents, pour leur amour inconditionnel, leur soutien sans faille et leurs encouragements constants tout au long de ce parcours.

Je pense également à tous mes amis de l'EAMAC, et plus spécialement à ceux qui me sont proches, pour leur présence, leur motivation et les moments partagés.

Je dédie aussi le projet pour la promotion EAC-SEI 2023 A-B et une mention spéciale à mon binôme **AVOCEGAMOU Lionel N.** dont la persévérance, l'expertise et la dévotion ont grandement contribué à l'aboutissement de ce travail.

RABARY Haja Tahina

# Remerciements

Avant toute chose, nous souhaitons exprimer notre profonde gratitude à Dieu le Tout-Puissant, dont la bienveillance nous a accompagnés tout au long de ce projet, en nous accordant la santé, la lucidité d'esprit et la force nécessaire pour mener ce travail à son terme.

Nous addressons également nos sincères remerciements à :

- Monsieur **HOUENOU Germain,** Chef de Division Electronique Informatique, pour ses conseils précieux et la rigueur pédagogique dont il a fait preuve à notre égard ;
- Monsieur **LAWALY Salifou** et Monsieur **TOLDE NGARADOUM**, responsables des promotions EAC SEI 2023 A - B, pour leur engagement constant, leur disponibilité et les conseils judicieux prodigués tout au long de notre formation ;
- Monsieur **DAKO Komlan Dominique Tavio**, instructeur à la Division Électronique et Informatique de l'EAMAC, pour son encadrement rigoureux, ses orientations éclairées et la bienveillance avec laquelle il a suivi l'évolution de ce projet ;
- Monsieur **MOUSSA MAMANE Rabiou**, instructeur à la Division Électronique et Informatique de l'EAMAC, pour l'inestimable apport qu'il a su offrir à notre projet, fort de ses compétences remarquables en électronique.

Nous adressons enfin notre reconnaissance à l'ensemble des instructeurs de l'EAMAC, dont la qualité irréprochable des cours dispensés, la rigueur pédagogique et l'esprit de collaboration ont fortement contribué à notre formation, ainsi qu'à tous les agents de l'établissement pour leur appui discret mais précieux.

# Résumé

Ce projet s'inscrit dans une démarche d'automatisation de précision, visant à concevoir et à réaliser un dispositif de calage automatique de l'angle d'inclinaison d'un plan. L'objectif est de détecter l'orientation angulaire d'un plan dans l'espace et de corriger dynamiquement sa position jusqu'à concordance avec une valeur de consigne prédéfinie par l'utilisateur.

Le système repose sur l'intégration d'un microcontrôleur ESP32 et d'un capteur inertiel MPU6050 combinant accéléromètre et gyroscope, permettant une mesure fiable et en temps réel de l'inclinaison. L'angle cible peut être renseigné soit manuellement via un potentiomètre, soit à distance par communication Bluetooth. Le dispositif assure ensuite les ajustements mécaniques à l'aide d'un servomoteur (pour l'axe de site) et d'un moteur pas à pas (pour l'axe d'azimut), commandés automatiquement en fonction de l'écart angulaire mesuré. L'ensemble est complété par un affichage clair des informations sur un écran LCD I2C 20×4.

Après une phase de conception virtuelle et de simulation à l'aide du logiciel Proteus, une réalisation pratique a été menée, validant la robustesse du système. Malgré certaines contraintes, notamment liées à la disponibilité des composants, les adaptations apportées ont permis de mener à bien l'implémentation.

Les perspectives d'application sont prometteuses, notamment dans le domaine aéronautique pour le réglage automatisé des unités d'un système PAPI, des feux d'approche haute intensité et feux flash, ainsi que dans le secteur de l'énergie solaire pour l'optimisation de l'inclinaison des panneaux photovoltaïques en fonction de l'ensoleillement (tracking du soleil).

# Abstract

This project is part of a broader effort to automate precise positioning systems and focuses on the design and implementation of an automatic tilt-angle alignment device for a given plane. The main objective is to detect the spatial orientation of a surface and to dynamically adjust its position until it matches a predefined target angle set by the user.

The developed system integrates an ESP32 microcontroller with an MPU6050 inertial sensor, combining a 3-axis accelerometer and gyroscope to ensure accurate real-time inclination measurements. The target angle can be entered either manually via a potentiometer or remotely through Bluetooth communication. Corrections are executed by a servomotor (site axis) and a stepper motor (azimuth axis), both automatically controlled based on the measured angular deviation. A 20×4 I2C LCD screen provides local visual feedback.

Following an initial design and simulation phase using Proteus software, the system was physically built and successfully tested, demonstrating its reliability and practical feasibility. Despite challenges such as component availability, iterative adjustments enabled the successful implementation of the prototype.

The application prospects are promising, particularly in the aerospace field for the automated adjustment of units of a PAPI system, high-intensity approach lights and flashing lights, as well as in the solar energy sector for optimizing the tilt of photovoltaic panels based on sunlight (sun tracking).

# Table des matières

**Dédicace** i

**Remerciements** ii

**Résumé** iii

**Abstract** iv

**Liste des Sigles** viii

**Cahier de Charge** xii

**Introduction** 1

## GÉNÉRALITÉ SUR LES COMPOSANTS

**1 Présentation de la Plateforme Arduino** 3

1.1. Architecture d'un microcontrôleur . . . 3

1.1.1. Unité centrale de traitement (CPU) . . . 3

1.1.2. Mémoire (ROM/Flash, RAM, EEPROM) . . . 4

1.1.3. Convertisseurs (ADC et DAC) . . . 4

1.1.4. Ports d'entrée/sortie (GPIO) . . . 5

1.1.5. Timers et compteurs (dont PWM) . . . 5

1.1.6. Interfaces de communication (UART, SPI, I2C,...) . . . 5

1.1.7. Bloc horloge (oscillateur, prescaler, PLL) . . . 5

1.1.8. Bus interne (données, adresses, contrôle) . . . 6

1.1.9. Système de gestion de l'alimentation . . . 6

1.2. Fonctionnement général de la carte Arduino . . . 6

1.2.1. La carte Arduino Uno . . . 6

1.2.2. La carte Arduino Mega . . . 7

1.3. La carte ESP32 . . . 8

1.4. IDE Arduino . . . 9

**2 Environnement de simulation : Proteus** 11

2.1. Présentation de Proteus . . . 11

2.2. Notions de simulation électronique . . . 12

**3 Composants électroniques et électromécaniques** 13

3.1. Rappels de base sur quelques composants électroniques . . . 13

3.1.1. Résistances . . . 13

3.1.2. Résistance variable . . . 14
3.1.3. Transistors . . . 15
3.1.4. Ecran LCD 16 × 2 . . . 15
3.1.5. La breadboard . . . 16
3.2. Particularités des composants électromécaniques utiles au projet . . . 17
3.2.1. Moteur DC . . . 17
3.2.2. Moteur pas à pas . . . 17
3.2.3. Servomoteur . . . 18
3.2.4. Vérin . . . 19

**4 Etude des Capteurs** **20**
4.1. Généralité sur les capteurs . . . 20
4.1.1. Définition et caractéristique générale . . . 20
4.1.2. Classification des capteurs . . . 21
4.1.3. Principe de fonctionnement des capteurs . . . 23
4.2. Les capteurs d'inclinaison . . . 24
4.2.1. Définition . . . 24
4.2.2. Principe de fonctionnement . . . 24
4.2.3. Les types de capteurs d'inclinaison . . . 25
4.2.4. Le capteur d'inclinaison MPU6050 . . . 26

## CONCEPTION ET SIMULATION DU SYSTÈME

**5 Description fonctionnelle du système** **29**
5.1. Objectif du système . . . 29
5.2. Logique de fonctionnement du système . . . 29
5.3. Choix des composants . . . 30
5.4. Schéma du montage sur Proteus . . . 31
5.5. Branchement des composants . . . 31
5.5.1. Connexion du potentiomètre à la carte arduino uno . . . 31
5.5.2. Connexion du servomoteur à la carte arduino uno . . . 32
5.5.3. Connexion de l'écran LCD à la carte arduino uno . . . 32
5.6. Analyse du Code Arduino pour la Simulation sous Proteus . . . 33
5.6.1. Explication des mots-clés du code . . . 35

## RÉALISATION PRATIQUE DU DISPOSITIF

**6 Implémentation réelle du circuit** **39**
6.1. Choix et acquisition des composants . . . 39
6.1.1. Liste du matériel utilisé . . . 39
6.1.2. Justification du choix des composants . . . 40

6.1.3. Schéma de cablage réel . . . . . 41
6.2. Code arduino exploité pour le montage . . . . . 42
6.2.1. Explication des mots-clés du code . . . . . 48
6.3. Evaluation des coûts . . . . . 49
6.4. Difficultés rencontrées . . . . . 50
6.5. Perspectives d'améliorations . . . . . 50

**7 Applications** **52**
7.1. Le PAPI . . . . . 52
7.1.1. Définition et fonctionnement . . . . . 52
7.1.2. Apport du système au PAPI . . . . . 53
7.2. Les panneaux photovoltaïques . . . . . 53
7.2.1. Définition et fonctionnement . . . . . 54
7.2.2. Apport du système au power tracking . . . . . 54

**Conclusion** **55**

**Bibliographie** **56**

# Liste des Sigles

| Sigle | Définition |
|---|---|
| **ADC** | Analog to Digital Converter (Convertisseur Analogique-Numérique) |
| **CPU** | Central Processing Unit (Unité centrale de traitement) |
| **DAC** | Digital to Analog Converter (Convertisseur Numérique-Analogique) |
| **DMP** | Digital Motion Processor (Processeur numérique de mouvement intégré au MPU6050) |
| **EEPROM** | Electrically Erasable Programmable Read-Only Memory (Mémoire non volatile réinscriptible) |
| **ESP32** | Module SoC (System on Chip) avec Wi-Fi et Bluetooth intégrés (Espressif) |
| **GPIO** | General Purpose Input/Output (Ports d'entrée/sortie génériques) |
| **I2C** | Inter-Integrated Circuit (Interface série à deux fils avec adressage) |
| **IDE** | Integrated Development Environment (Environnement de développement intégré) |
| **IMU** | Inertial Measurement Unit (Unité de mesure inertielle) |
| **LCD** | Liquid Crystal Display (Afficheur à cristaux liquides) |
| **LDR** | Light Dependent Resistor (Résistance dépendante de la lumière) |
| **MEMS** | Micro-Electro-Mechanical Systems (Systèmes micro-électromécaniques) |
| **MPU6050** | Module IMU combinant accéléromètre et gyroscope 3 axes |
| **PAPI** | Precision Approach Path Indicator (Indicateur de pente d'approche de précision) |
| **PLL** | Phase-Locked Loop (Boucle à verrouillage de phase) |
| **PWM** | Pulse Width Modulation (Modulation de largeur d'impulsion) |
| **RAM** | Random Access Memory (Mémoire vive) |
| **ROM** | Read-Only Memory (Mémoire en lecture seule) |
| **SPI** | Serial Peripheral Interface (Interface série synchrone) |
| **TOR** | Tout Ou Rien (Signal binaire : 0 ou 1) |
| **UART** | Universal Asynchronous Receiver/Transmitter (Interface série asynchrone) |

# Table des figures


1.1 Architecture du microcontrôleur . . . . . 4

1.2 Carte Arduino Uno . . . . . 7

1.3 Carte Arduino Mega . . . . . 7

1.4 La carte ESP-WROOM-32 . . . . . 8

1.5 Interface graphique Arduino . . . . . 10

2.1 Interface graphique de Proteus . . . . . 11

2.2 Exemple d'une simulation sur Proteus . . . . . 12

3.1 Photo de résistance . . . . . 13

3.2 Symbole de résistance . . . . . 13

3.3 Un potentiomètre . . . . . 14

3.4 Symbole de résistance variable . . . . . 14

3.5 Transistor . . . . . 15

3.6 Ecran lcd 16 × 2 . . . . . 16

3.7 Plaque d'essai . . . . . 16

3.8 Moteur DC . . . . . 17

3.9 Moteur pas à pas . . . . . 18

3.10 Servomoteur . . . . . 18

3.11 Image d'un vérin électrique . . . . . 19

4.1 Structure d'un capteur . . . . . 21

4.2 Fonctionnement d'un capteur . . . . . 23

4.3 Le capteur d'inclinaison ADXL345 . . . . . 24

4.4 Le capteur MPU6050 . . . . . 26

5.1 Schéma de la simulation . . . . . 31

6.1 Schéma du montage avec le logiciel fritzing . . . . . 41

6.2 Vue du montage finalisé . . . . . 42

7.1 Le PAPI . . . . . 53

7.2 Les panneaux photovoltaïques . . . . . 54

EAMAC

# Liste des tableaux


1.1 Caractéristiques principales de la carte ESP32 . . . . . 9

3.1 Code des couleurs . . . . . 14

4.1 Classification des capteurs selon la mesurande . . . . . 22

4.2 Classification des capteurs selon le type de signal de sortie . . . . . 22

4.3 Classification des capteurs selon le principe de fonctionnement . . . . . 23

4.4 Types de capteurs d'inclinaison . . . . . 25

4.5 Caractéristiques techniques essentielles du capteur MPU-6050 . . . . . 26

5.1 Composants utilisés et leur rôle dans la simulation . . . . . 30

5.2 Branchements entre le potentiomètre et la carte Arduino Uno . . . . . 32

5.3 Branchements entre le servomoteur et la carte Arduino Uno . . . . . 32

5.4 Branchements entre l'écran LCD et la carte Arduino Uno . . . . . 32

5.5 Résumé des mots-clés et fonctions utilisés dans le code Arduino . . . . . 36

6.1 Liste des composants utilisés pour le montage réel . . . . . 39

6.2 Résumé des mots-clés du code et leurs fonctions . . . . . 49

6.3 Tableau estimatif des coûts du prototype . . . . . 50

# Cahier de Charge

**INTITULE :** MISE EN OEUVRE DE CALAGE AUTOMATIQUE DE L'ANGLE D'INCLINAISON D'UN PLAN

**Encadrant :** M. DAKO Komlan Dominique Tavio

**CONTEXTE :**

Le calage des unités PAPI lors des opérations de calibration en vol nécessite parfois des corrections d'angle rapides à faire. Le temps d'intervention et de remise en service d'un système PAPI déclaré hors tolérance constitue des coûts énormes pour la maintenance et a des impacts opérationnels. Par ailleurs, les systèmes de poursuite de la puissance maximale d'une centrale solaire photovoltaïque sollicitent le suivi en temps réel de l'inclinaison des panneaux par rapport aux rayons du soleil.

La mise en œuvre de ce dispositif faciliterait les opérations de calibration et constitue une bonne base pour l'optimisation de la production des centrales solaires photovoltaïques.

**DESCRIPTION :**

Le projet consiste à l'étude de la mise en œuvre d'un module de lecture d'un angle affichée à l'écran, à la génération d'un signal électrique permettant de modifier l'angle en site et en azimut d'un plan donné.

**SPECIFICATIONS GENERALES :**

Le système sera bâti autour d'un microcontrôleur et composants divers (afficheur, composants électroniques, composants électromécaniques, ...).

**SPECIFICATIONS FONCTIONNELLES DU SYSTEME :**

Le dispositif doit permettre de caler en site et en azimut l'angle d'un plan saisi à l'écran. Le dispositif peut faire l'objet d'une réalisation pratique entièrement ou partiellement et/ou pourra être simulé.

**PERSPECTIVES :**

Le système mis en œuvre permettra de servir de base pour la conception d'un système de PAPI muni de vérin à commande automatique. Il pourra aussi être utilisé dans la conception d'un système de power tracking des centrales solaires photovoltaïques.

# Introduction

Dans un contexte où l'automatisation occupe une place croissante dans les systèmes techniques modernes, la recherche de solutions intelligentes et autonomes devient une priorité dans de nombreux domaines d'application. Grâce à la flexibilité offerte par les microcontrôleurs comme l'Arduino/ESP32, il est aujourd'hui possible de concevoir des dispositifs capables de s'adapter à leur environnement et d'agir en conséquence, tout en restant simples et économiques à mettre en œuvre. Dans ce cadre, le présent projet porte sur **la conception et la réalisation d'un système de calage automatique de l'angle d'inclinaison d'un plan**, capable de détecter et corriger son orientation de manière autonome. Ce type de dispositif trouve des applications concrètes dans des domaines tels que l'orientation de panneaux solaires ou les systèmes de guidage visuel dans l'aéronautique, où la précision angulaire est primordiale.

La démarche adoptée repose sur trois étapes complémentaires : une généralité sur les composants et outils mobilisés, une phase de conception et de simulation à l'aide du logiciel Proteus, puis une réalisation pratique du système à l'aide du microcontrôleur ESP32. Ce rapport rend compte de l'ensemble du processus de développement, des fondements conceptuels aux essais finaux, en mettant en évidence les choix techniques opérés ainsi que les perspectives d'évolution du dispositif.

# Première partie

# Généralité sur les Composants

Chapitre 1

# Présentation de la Plateforme Arduino

## 1.1. Architecture d'un microcontrôleur

Un microcontrôleur est un circuit intégré regroupant plusieurs blocs fonctionnels permettant le contrôle d'un système embarqué. Contrairement à un microprocesseur classique, il intègre directement la mémoire, les périphériques et les interfaces nécessaires à son fonctionnement [3]. L'architecture générale peut se diviser en plusieurs blocs principaux :

- Unité centrale de traitement (CPU),
- Mémoire (ROM/Flash, RAM, EEPROM),
- Convertisseurs (ADC et DAC),
- Ports d'entrée/sortie (GPIO),
- Timers et compteurs (dont PWM),
- Interfaces de communication (UART, SPI, I2C, etc.),
- Bloc horloge (oscillateur, prescaler, PLL),
- Bus interne (données, adresses, contrôle),
- Système de gestion de l'alimentation (régulateurs, modes basse consommation, détection/-protection).

### 1.1.1. Unité centrale de traitement (CPU)

L'unité centrale de traitement constitue le cœur logique du microcontrôleur : c'est elle qui orchestre l'exécution du programme. Elle abrite **l'Unité Arithmétique et Logique (ALU)**, responsable des opérations mathématiques et logiques, **le Registre d'Instruction (IR)**, qui stocke l'instruction en cours de décodage, et **le Compteur Ordinal (PC)**, chargé de pointer vers l'adresse de la prochaine instruction. L'efficacité de la CPU détermine directement les performances globales du système, notamment en termes de rapidité d'exécution et de latence lors des traitements critiques.

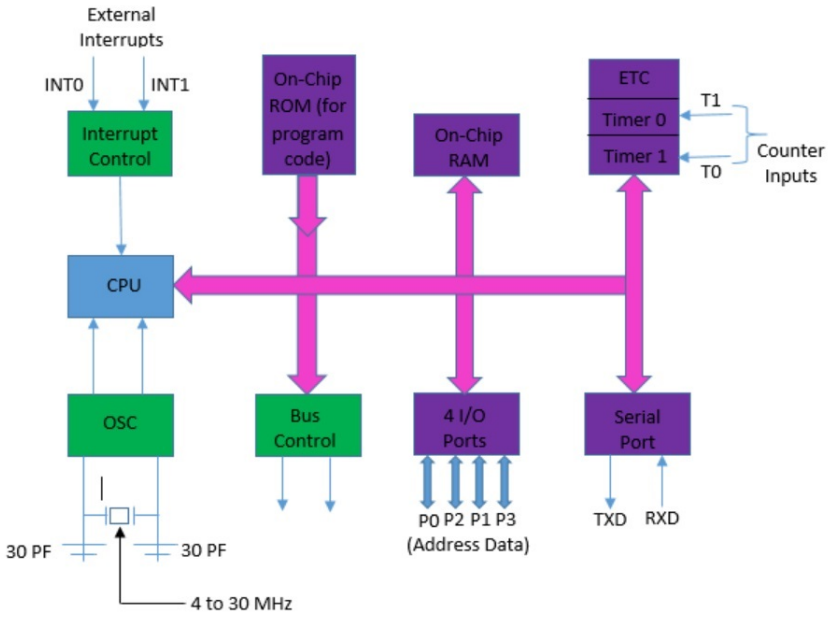



FIGURE 1.1 – Architecture du microcontrôleur

## 1.1.2. Mémoire (ROM/Flash, RAM, EEPROM)

Le bloc mémoire regroupe plusieurs types complémentaires : **la ROM**, non volatile, stocke de manière permanente des instructions ou données critiques dès la fabrication ; **la Flash**, également non volatile mais réinscriptible, héberge le firmware et facilite les mises à jour ; **la RAM**, volatile, sert de mémoire de travail pour les variables temporaires, la pile et les buffers, avec des accès très rapides pour ne pas ralentir la CPU ; enfin, **l'EEPROM**, non volatile et réinscriptible à l'octet, conserve paramètres utilisateur et journaux d'événements entre deux mises sous tension.

## 1.1.3. Convertisseurs (ADC et DAC)

Le bloc convertisseurs intègre d'une part un **ADC (Analog to Digital Converter)** qui traduit un signal analogique (capteur, tension variable…) en données numériques exploitables par la CPU, avec une résolution (8-12 bits) et une fréquence d'échantillonnage déterminantes, et d'autre part un **DAC (Digital to Analog Converter)** qui reconvertit des valeurs numériques en tensions analogiques, utile pour piloter actionneurs ou générer des formes d'ondes. Cette double fonctionnalité étend les capacités du microcontrôleur à l'acquisition et à la restitution de signaux variés.

### 1.1.4. Ports d'entrée/sortie (GPIO)

Ce sont des Interfaces permettant de communiquer avec des capteurs, actionneurs ou autres composants externes. Elles peuvent être numériques (GPIO), analogiques (ADC/DAC) ou de communication (UART, SPI, I2C...).

### 1.1.5. Timers et compteurs (dont PWM)

Les timers/counters fournissent des fonctions essentielles pour la gestion temporelle : mesure de durée, génération de délais précis, comptage d'événements externes, et création de signaux PWM (Pulse-Width Modulation). On peut, par exemple, programmer un timer pour déclencher une interruption périodique ou pour piloter avec finesse la luminosité d'une LED via PWM, ou encore mesurer la fréquence d'un signal d'entrée.

### 1.1.6. Interfaces de communication (UART, SPI, I2C,...)

Le bloc Interfaces de communication d'un microcontrôleur permet l'échange de données entre le microcontrôleur et des périphériques externes (capteurs, mémoires, afficheurs, etc.). Il regroupe plusieurs protocoles standards tels que UART, SPI, et I2C, chacun ayant ses spécificités.
L'UART assure une communication série asynchrone via deux fils (TX, RX) sans signal d'horloge, utilisé notamment pour les liaisons simples comme la console série ou le Bluetooth.
Le SPI offre une communication synchrone rapide avec plusieurs lignes (MOSI, MISO, SCLK, SS), idéal pour les échanges à haute vitesse avec des mémoires ou des écrans.
Quant à l'I2C, il utilise seulement deux fils (SDA, SCL) pour relier plusieurs périphériques grâce à un adressage unique.

### 1.1.7. Bloc horloge (oscillateur, prescaler, PLL)

Le bloc horloge génère le signal de synchronisation du microcontrôleur, soit via un **oscillateur interne** rapide et économique, soit via un **cristal externe** plus stable. Un diviseur d'horloge (et parfois un PLL) ajuste ou multiplie la fréquence pour alimenter séparément la CPU et les périphériques.
Le choix de la source et du réglage d'horloge influence directement la réactivité, la précision des communications et la consommation énergétique.

### 1.1.8. Bus interne (données, adresses, contrôle)

Le bus interne assure la communication entre CPU, mémoire et périphériques : le **bus de données** transfère l'information, le **bus d'adresses** en précise la source ou la destination, et le **bus de contrôle** coordonne lectures, écritures et interruptions. Sa largeur (8 à 32 bits) détermine le volume de données échangé par cycle d'horloge.

### 1.1.9. Système de gestion de l'alimentation

Le bloc de gestion d'alimentation comprend des régulateurs interne et externe pour stabiliser la tension d'entrée, ainsi que des modes basse consommation (veille, arrêt) afin d'optimiser l'autonomie. Il intègre aussi des circuits de détection et de protection contre les sous-tensions et surtensions, garantissant la fiabilité du microcontrôleur tout en limitant la consommation énergétique.

## 1.2. Fonctionnement général de la carte Arduino

Des cartes Arduino, il en existe beaucoup, peut etre une centaine toutes différentes. Ici, nous présentons principalement deux cartes : la carte Arduino Uno et la carte Arduino Mega.

### 1.2.1. La carte Arduino Uno

L'Arduino Uno est une carte de prototypage open-source basée sur le microcontrôleur ATmega-328P. Elle dispose d'une interface USB-B pour la programmation et l'alimentation 5 V depuis un PC, ainsi que d'une prise jack 7–12 V pour une alimentation externe. Un régulateur embarqué abaisse cette tension aux niveaux 5 V et 3,3 V disponibles sur la broche 5V, la broche 3V3 et plusieurs broches GND.
Sur les côtés de la carte, on trouve **14 broches numériques (D0–D13)** dont 6 peuvent délivrer un signal PWM, ainsi que **6 entrées analogiques (A0–A5)** reliées à l'ADC du microcontrôleur. Chaque broche numérique peut être configurée en entrée/sortie, prendre en charge des résistances pull-up ou des interruptions sur front, tandis que les broches analogiques mesurent des tensions de 0 à 5 V avec une résolution de 10 bits.
Le cœur de la carte est protégé par un bouton **RESET**, qui relance le programme stocké en mémoire Flash. Un second microcontrôleur (ATmega16U2) gère la conversion USB-série, simplifiant la communication entre le PC et le module principal. Le quartz 16 MHz assure la cadence d'exé-

cution, tandis qu'un petit circuit EEPROM permet de conserver des paramètres utilisateur indépendamment du programme principal.[2, 4]

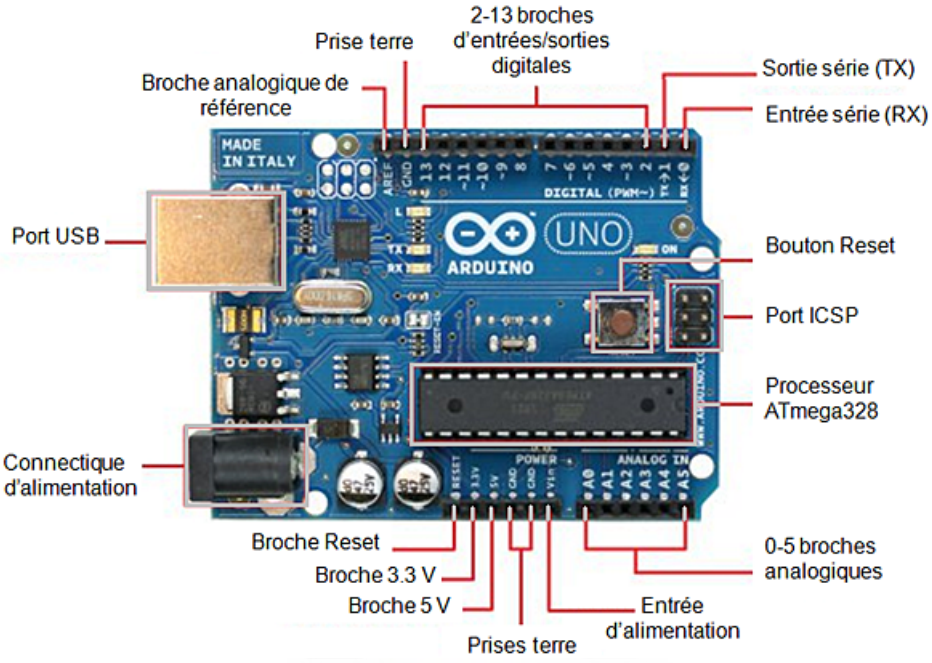



FIGURE 1.2 – Carte Arduino Uno

## 1.2.2. La carte Arduino Mega

L'Arduino Mega 2560 est une carte plus puissante et plus extensible que l'Arduino Uno, idéale pour les projets complexes. Contrairement à l'Uno, elle offre 54 broches numériques (contre 14), 16 entrées analogiques (contre 6), 4 ports série (contre 1) et une mémoire flash de 256 Ko (contre 32 Ko), ce qui la rend mieux adaptée aux applications nécessitant de nombreux capteurs, modules ou un code volumineux.[2, 4]

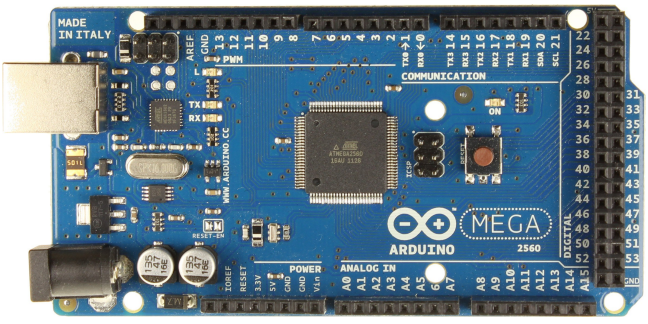


FIGURE 1.3 – Carte Arduino Mega

## 1.3. La carte ESP32

La ESP32 est une carte de développement basée sur un microcontrôleur SoC (System on Chip) fabriqué par Espressif Systems. Elle est conçue pour des applications embarquées et connectées, notamment dans les domaines de l'IoT (Internet of Things), de la domotique, de la robotique et de l'automatisation industrielle.

Ce microcontrôleur se distingue par sa double connectivité Wi-Fi et Bluetooth, sa puissance de traitement, son nombre élevé d'entrées/sorties, et sa faible consommation d'énergie. Il est largement utilisé avec des environnements de développement comme Arduino IDE, PlatformIO ou ESP-IDF.




FIGURE 1.4 – La carte ESP-WROOM-32

Le tableau ci-dessous présente les caractéristiques techniques [6] essentielles de la carte ESP32.

| Caractéristique | Description |
|---|---|
| Microcontrôleur | ESP32-D0WDQ6 (ou variante selon la carte) |
| Architecture | Dual-core Xtensa LX6 (jusqu'à 240 MHz) |
| Mémoire RAM | Environ 520 Ko SRAM interne |
| Mémoire Flash | De 4 Mo à 16 Mo selon les modèles |
| Wi-Fi | IEEE 802.11 b/g/n, mode station/AP/dual |
| Bluetooth | Bluetooth v4.2 BR/EDR + BLE |
| GPIO | Jusqu'à 34 broches GPIO (fonction multiplexée) |
| Entrées analogiques | 18 entrées ADC 12 bits (sur certaines broches) |
| Sorties analogiques | 2 sorties DAC 8 bits |
| PWM | Génération de signaux PWM sur la plupart des GPIO |
| Interfaces de communication | UART (3), SPI (4), I2C (2), I2S, CAN, IR, PWM |
| Sécurité | Cryptographie matérielle (AES, SHA, RSA, etc.) |
| Consommation | Ultra-basse consommation avec modes Deep Sleep |
| Tension d'alimentation | 3.3 V (via régulateur à 5 V en entrée) |
| Débogage | Interface UART via USB ou JTAG |
| Compatibilité IDE | Arduino IDE, ESP-IDF, PlatformIO, Micropython, etc. |
| Dimensions typiques | Environ 25 mm x 50 mm (varie selon le module) |

TABLE 1.1 – Caractéristiques principales de la carte ESP32

## 1.4. IDE Arduino

L'IDE Arduino (Integrated Development Environment) (ou Environnenment de Développement Intégré en français) est le logiciel officiel utilisé pour programmer toutes les cartes Arduino, y compris l'Arduino Uno et la Mega 2560. Simple et intuitif, il permet d'écrire, compiler et téléverser du code en langage C/C++ vers la carte via un câble USB. Il intègre un éditeur de code, un moniteur série pour visualiser les données en temps réel, un traceur série pour afficher graphiquement les valeurs, ainsi qu'un gestionnaire de bibliothèques facilitant l'ajout de composants comme les capteurs, écrans ou modules de communication.
Conçu aussi bien pour les débutants que pour les projets avancés, l'IDE offre un environnement complet et accessible pour le développement embarqué.[2, 1]

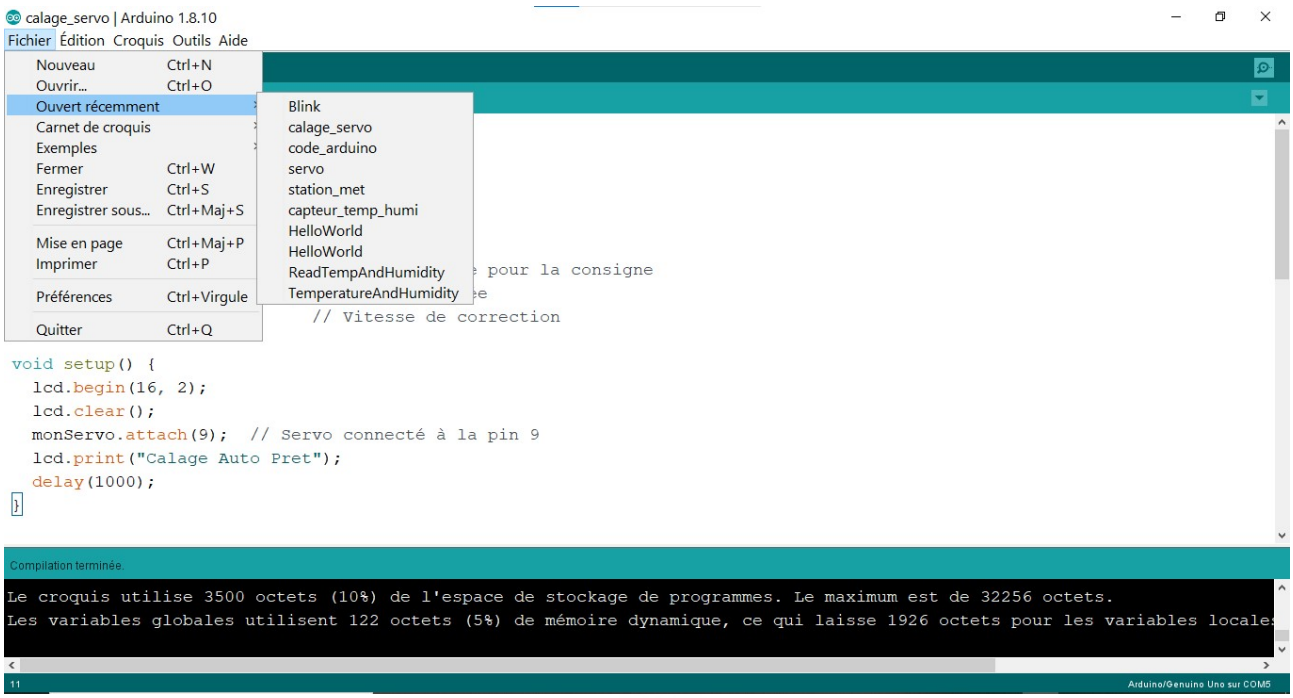



FIGURE 1.5 – Interface graphique Arduino

Chapitre 2

# Environnement de simulation : Proteus

## 2.1. Présentation de Proteus

Proteus est un logiciel polyvalent de conception et de simulation électronique qui permet non seulement de réaliser des schémas et de concevoir des circuits imprimés, mais aussi de simuler en temps réel des circuits analogiques, numériques et embarqués, notamment avec des microcontrôleurs comme Arduino.

Très apprécié pour sa capacité à tester les programmes et montages sans matériel physique, il propose une large bibliothèque de composants et une interface conviviale, idéale tant pour les étudiants que les professionnels.

Cependant, Proteus est un logiciel payant avec une version d'essai limitée, certaines bibliothèques doivent être ajoutées manuellement, et sa précision peut diminuer pour des circuits très complexes ou à haute fréquence. Malgré ces quelques contraintes, il demeure un outil puissant et fiable pour valider et optimiser efficacement des projets électroniques avant leur réalisation.

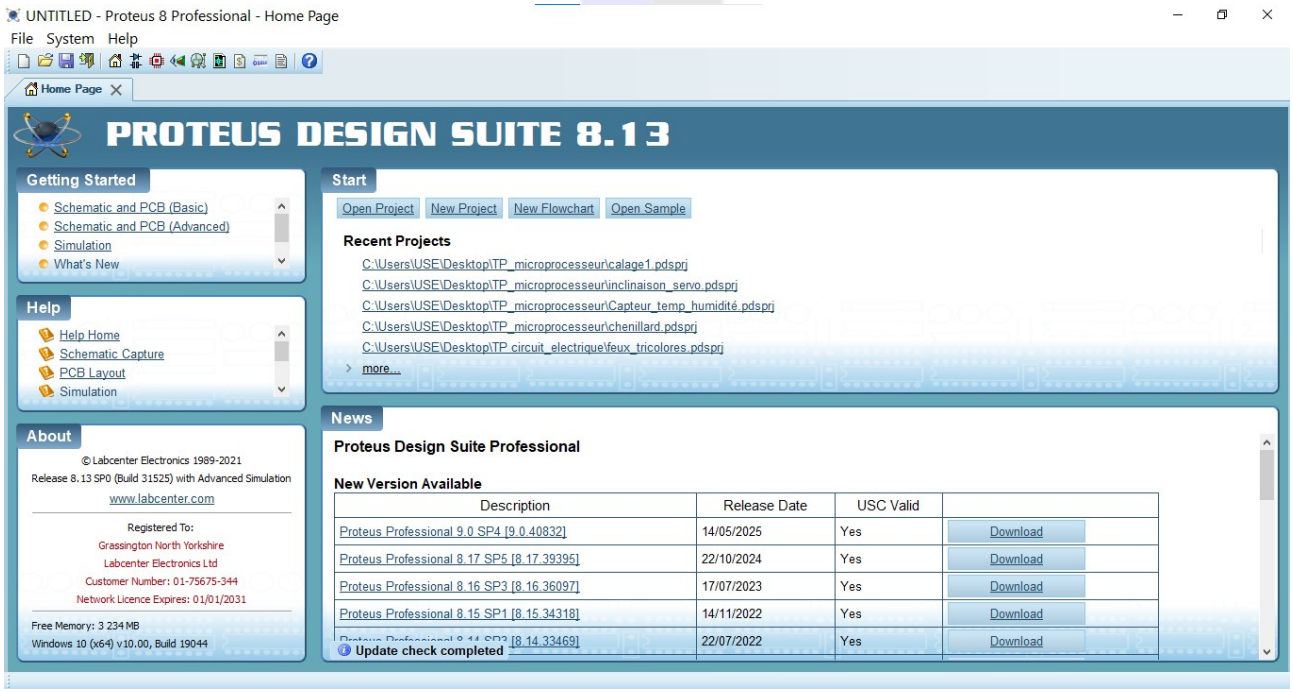



FIGURE 2.1 – Interface graphique de Proteus

## 2.2. Notions de simulation électronique

La simulation électronique est un processus qui consiste à reproduire virtuellement le comportement d'un circuit électronique à l'aide d'un logiciel spécialisé (Proteus par exemple), sans avoir besoin de construire physiquement le montage.
Grâce à cette technique, il est possible d'analyser le fonctionnement des composants (résistances, transistors, microcontrôleurs, etc.), de vérifier les interactions entre eux, de détecter des erreurs ou des défauts de conception, et d'optimiser le circuit avant sa réalisation matérielle.
La simulation permet ainsi de gagner du temps, de réduire les coûts et d'améliorer la fiabilité des projets électroniques en anticipant les problèmes potentiels dès la phase de conception.

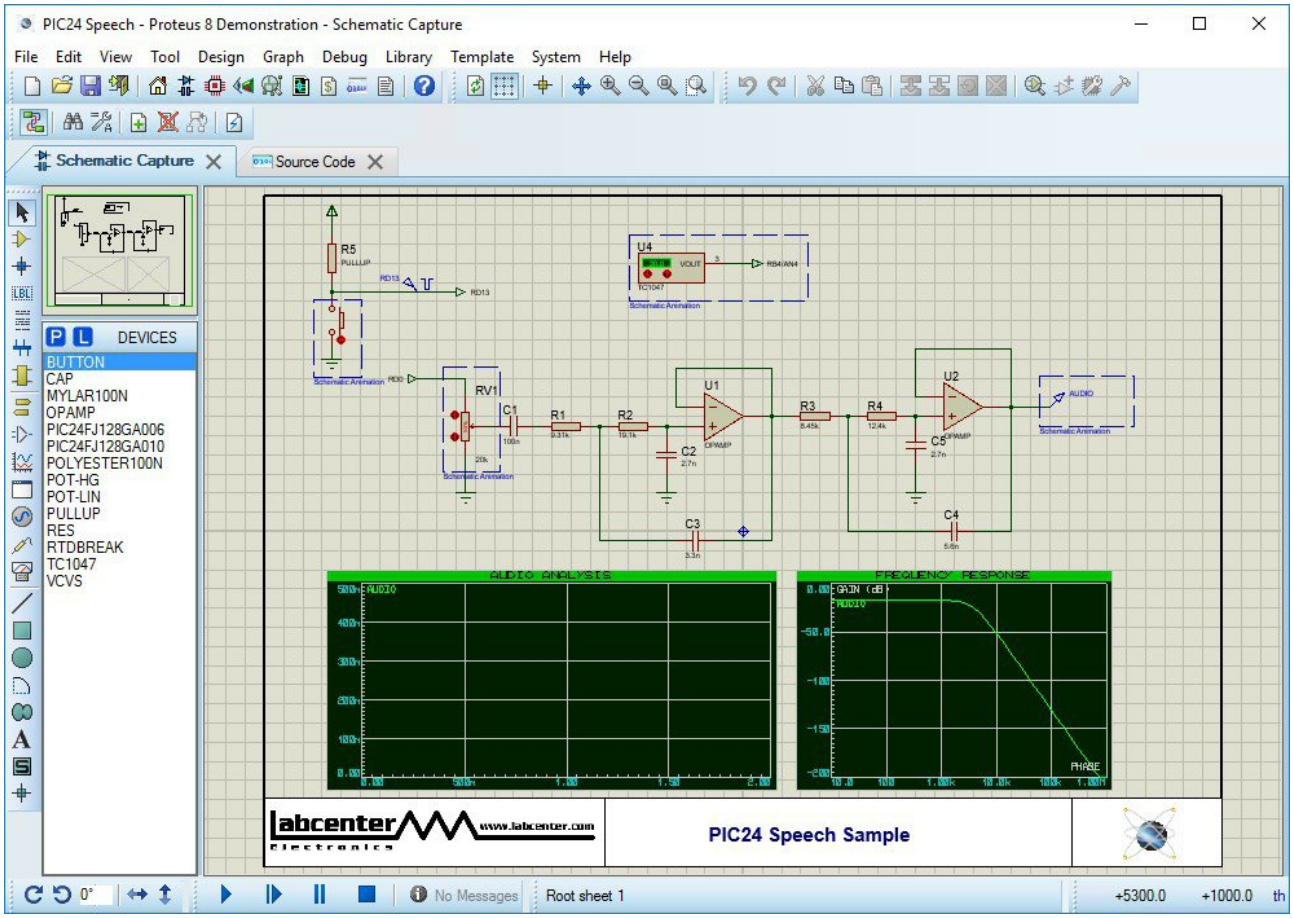



FIGURE 2.2 – Exemple d'une simulation sur Proteus

Chapitre 3

# Composants électroniques et électromécaniques

## 3.1. Rappels de base sur quelques composants électroniques

### 3.1.1. Résistances

C'est le composant le plus utilisé en électronique. Sa principale fonction est de réduire l'intensité du courant. Ce composant se présente sous la forme d'un petit boitier fait de divers matériaux et repéré par des anneaux de couleur indiquant la valeur de cette dernière.[2]



FIGURE 3.1 – Photo de résistance

Le symbole de la résistance ressemble étrangement à la forme de son boitier :

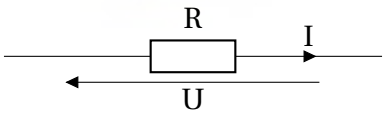



FIGURE 3.2 – Symbole de résistance

Le courant traversant une résistance est régi par une formule assez simple, qui se nomme la loi d'ohm : $I = \frac{U}{R}$ avec
$I$ : intensité qui traverse la résistance en Ampères, notée $A$.
$U$ : tension aux bornes de la résistance en Volts, notée $V$.
$R$ : valeur de la résistance en Ohms, notée Ω.
En général, on retient mieux la formule sous cette forme : $U = R \times I$.
L'unité de la résistance est l'ohm. On le note avec le symbole oméga en majuscule Ω.
Le code des couleurs : La résistance possède une suite d'anneaux de couleurs différentes sur son boitier. Ce tableau vous permettra de lire ce code qui correspond à la valeur de la résistance.

| Couleur | Chiffre | Coefficient multiplicateur | Puissance | Tolérance |
|---|---|---|---|---|
| Noir | 0 | 1 | $10^0$ | – |
| Brun | 1 | 10 | $10^1$ | ±1% |
| Rouge | 2 | 10 | $10^2$ | ±2% |
| Orange | 3 | 1000 | $10^3$ | – |
| Jaune | 4 | 10000 | $10^4$ | – |
| Vert | 5 | 100000 | $10^5$ | ±0.5% |
| Bleu | 6 | 1000000 | $10^6$ | ±0.25% |
| Violet | 7 | 10000000 | $10^7$ | ±0.10% |
| Gris | 8 | 100000000 | $10^8$ | ±0.05% |
| Blanc | 9 | 1000000000 | $10^9$ | – |
| – | – | – | – | – |
| Or | 0.1 | 0.1 | $10^{-1}$ | ±5% |
| Argent | 0.01 | 0.01 | $10^{-2}$ | ±10% |
| (absent) | – | – | – | ±20% |

TABLE 3.1 – Code des couleurs

### 3.1.2. Résistance variable

Une résistance variable est un composant électronique dont la valeur de résistance peut être ajustée manuellement ou automatiquement, permettant ainsi de contrôler le courant électrique dans un circuit, ce qui est essentiel pour ajuster la performance du système.
Les types les plus courants sont :

- les potentiomètres, composés de trois bornes, avec un curseur mobile qui se déplace le long d'une piste résistive. En modifiant la position du curseur, on change la longueur effective de la piste résistive entre les bornes, ce qui ajuste la résistance [11] ;
- les trimmers sont des petites résistances variables destinées à effectuer des réglages fins permanents à l'intérieur des appareils électroniques.
  Contrairement aux potentiomètres accessibles à l'utilisateur, les trimmers sont souvent ajustés lors de la fabrication ou de la maintenance du dispositif.



FIGURE 3.3 – Un potentiomètre

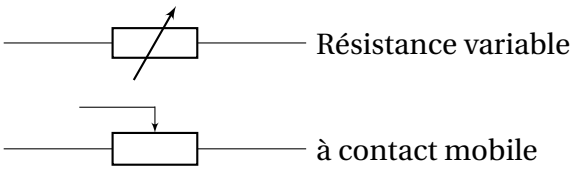



FIGURE 3.4 – Symbole de résistance variable

### 3.1.3. Transistors

Les transistors sont des dispositifs semi-conducteurs utilisés pour amplifier ou commuter des signaux électriques. Ils se classent en deux grandes catégories : les transistors bipolaires ( à trois broches : base-collecteur-émetteur) et les transistors à effet de champ (FET) égalemnt à trois broches ( grille-drain-source).
Son principe de fonctionnement repose sur la commande d'un courant (pour le transistor bipolaire) ou d'une tension (pour le transistor FET) faible à la broche de commande pour laisser passer un courant plus important entre les deux autres broches.
générées à l'arrêt.
Les transistors bipolaires existent en deux types, NPN et PNP, qui diffèrent par le sens des courants et des tensions.

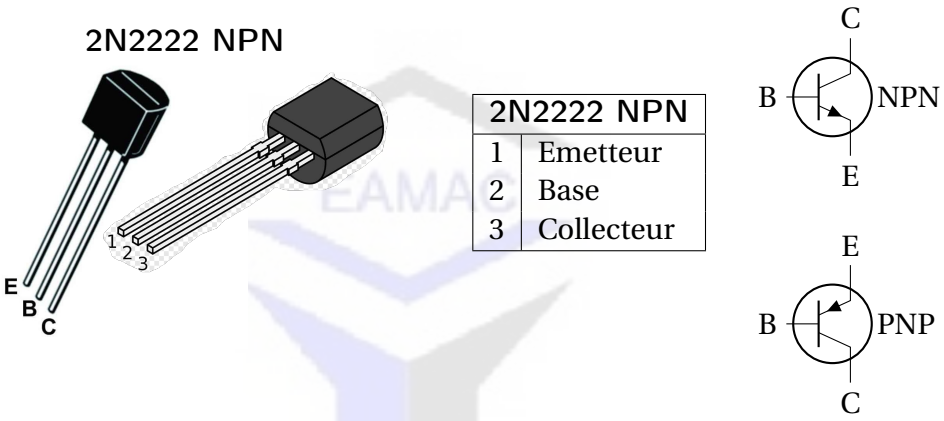



FIGURE 3.5 – Transistor

### 3.1.4. Ecran LCD $16 \times 2$

Un écran LCD 16×2 est un afficheur alphanumérique basé sur le contrôleur HD44780 (ou compatible) capable d'afficher jusqu'à 16 caractères par ligne sur deux lignes indépendantes.
Chaque caractère est formé par une matrice de points 5×8 permettant de représenter lettres, chiffres et symboles de base [10].
Le module comprend généralement un rétroéclairage (LED derrière l'écran) et un potentiomètre de contraste ajustable. Physiquement, on le repère à ses 16 broches (généralement deux rangées de 8), dont les principales sont :

- VSS (masse), VDD (+5 V d'alimentation) et V0 (entrée du potentiomètre pour le contraste).
- *RS* (Register Select) pour basculer entre mode commande (instructions) et mode données (caractères).
- *RW* (Read/Write), qui est souvent relié à la masse (mode écriture uniquement).
- *E* (Enable) pour valider la lecture ou l'écriture d'un octet sur le bus de données.

- $D0$–$D7$ (bus de données 8 bits) ou $D4$–$D7$ (bus réduit 4 bits si on choisit un mode de communication sur 4 fils).

Une résistance de limitation pour la LED de rétroéclairage peut être présente sur le circuit ou être ajoutée séparément selon le modèle.



FIGURE 3.6 – Ecran lcd 16 × 2

### 3.1.5. La breadboard

La breadboard, ou plaque d'essai, est un dispositif utilisé en électronique pour assembler temporairement des circuits sans recours à la soudure. Elle se présente sous la forme d'une plaque en plastique perforée (percée de nombreux trous), organisés de manière à permettre l'insertion facile de composants électroniques.
À l'intérieur, des bandes métalliques assurent les connexions électriques entre les points de contact. La partie centrale est souvent séparée par une tranchée destinée à accueillir des circuits intégrés sans que leurs broches ne se touchent. Des rails d'alimentation situés sur les bords permettent de distribuer la tension et la masse à l'ensemble du circuit.
Le principe de fonctionnement repose donc sur une interconnexion simple, rapide et modifiable des composants, ce qui rend la breadboard particulièrement utile pour les phases de test et d'expérimentation [2].

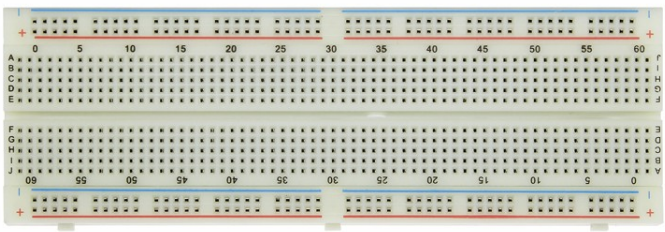


FIGURE 3.7 – Plaque d'essai

## 3.2. Particularités des composants électromécaniques utiles au projet

### 3.2.1. Moteur DC

Un moteur à courant continu (moteur DC) est un actionneur électromécanique qui convertit une tension continue en un mouvement de rotation. Il est constitué principalement d'un rotor (ou armature) bobiné, qui tourne à l'intérieur d'un champ magnétique généré par le stator, constitué soit d'aimants permanents, soit d'électroaimants (bobines fixes). Lorsque l'on applique une tension entre les bornes du moteur, un courant traverse les enroulements du rotor, induisant un champ magnétique. L'interaction entre ce champ et celui du stator génère un couple électromagnétique qui entraîne la rotation de l'arbre. La vitesse de rotation est généralement proportionnelle à la tension d'alimentation, tandis que le couple produit est proportionnel au courant consommé, lequel augmente notamment lors du démarrage ou en présence d'une charge mécanique.

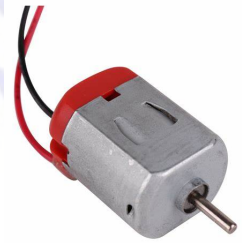


FIGURE 3.8 – Moteur DC

### 3.2.2. Moteur pas à pas

Les moteurs pas à pas sont des moteurs électriques synchrones conçus pour convertir des signaux électriques en déplacements mécaniques discrets. Contrairement aux moteurs à courant continu classiques qui tournent de manière continue lorsqu'ils sont alimentés, les moteurs pas à pas fonctionnent par mouvements angulaires successifs appelés « pas ». Chaque impulsion électrique envoyée à l'un de ses enroulements permet au rotor d'effectuer un angle précis, ce qui en fait un dispositif de choix lorsqu'un contrôle rigoureux de la position est requis.
Le moteur pas à pas fonctionne en faisant tourner son rotor par incréments angulaires fixes, appelés pas, grâce à l'activation séquentielle de ses enroulements statoriques. À chaque impulsion électrique envoyée à une bobine spécifique, le champ magnétique créé attire le rotor vers une position stable. En envoyant ces impulsions dans un ordre précis, le rotor avance pas à pas. Le nombre total de pas nécessaires pour effectuer un tour complet dépend du type de moteur (par

exemple : 200 pas/tour équivalent à 1,8° par pas). Ce fonctionnement permet un contrôle précis de la position et de la vitesse, sans capteur de position.
Il existe plusieurs types de moteurs pas à pas, les plus courants étant les moteurs à aimant permanent, à réluctance variable et hybrides. Leur fonctionnement nécessite l'utilisation d'un circuit de commande adapté, appelé driver (comme l'ULN2003, le A4988 ou le DRV8825), capable de générer les séquences logiques nécessaires au déplacement du rotor. Un microcontrôleur tel qu'un Arduino est souvent utilisé pour envoyer les impulsions de commande [8, 9].
Le moteur pas à pas offre un positionnement précis, un bon couple à basse vitesse et un contrôle simple sans capteur, mais il consomme même à l'arrêt, perd en efficacité à haute vitesse et nécessite une commande plus complexe.



FIGURE 3.9 – Moteur pas à pas

### 3.2.3. Servomoteur

Un servomoteur (ou « servo ») est un système tout-en-un qui combine un petit moteur (généralement un moteur DC à balais ou brushless), un réducteur mécanique, un capteur de position (potentiomètre ou encodeur), et l'électronique de commande embarquée. Le principe de base consiste à recevoir un signal PWM standard (largeur d'impulsion comprise entre $1ms$ et $2ms$, généralement envoyé depuis un microcontrôleur), chaque largeur d'impulsion correspondant à un angle précis (par exemple de 0 ° à 180 °). Le contrôleur interne compare constamment la position réelle, mesurée par le capteur, à l'angle souhaité et ajuste la tension appliquée au moteur pour réduire l'écart, de façon à atteindre puis maintenir la position demandée [7].
Il est idéal pour les applications où la précision et la stabilité sont essentielles, comme les bras robotiques ou les systèmes de visée. Toutefois, il offre un angle de rotation limité (souvent 180°), un couple généralement faible pour les petits modèles, et son prix peut être plus élevé que celui d'un simple moteur DC.



FIGURE 3.10 – Servomoteur

### 3.2.4. Vérin

Un vérin est un actionneur linéaire qui convertit une forme d'énergie (électrique, pneumatique ou hydraulique) en un mouvement de translation. Contrairement aux moteurs qui produisent un mouvement rotatif, les vérins sont utilisés lorsqu'on a besoin de pousser, tirer ou lever une charge sur une certaine distance. Ils sont largement utilisés dans les domaines industriels, robotiques, automobiles, et dans l'automatisation en général.
Le fonctionnement d'un vérin repose sur la conversion d'une énergie (hydraulique, pneumatique ou électrique) en un mouvement linéaire. On distingue principalement trois types de vérins selon l'énergie utilisée.

- Le vérin hydraulique utilise un liquide sous pression (généralement de l'huile) pour déplacer un piston à l'intérieur d'un cylindre ; il est adapté aux applications nécessitant une grande force.
- Le vérin pneumatique, quant à lui, fonctionne avec de l'air comprimé et est souvent utilisé dans les systèmes industriels pour sa rapidité et sa propreté, bien qu'il soit moins puissant.
- Le vérin électrique transforme le mouvement rotatif d'un moteur en mouvement linéaire à l'aide d'un mécanisme à vis (vis sans fin ou tige filetée) ; il se distingue par sa précision et sa facilité d'automatisation, mais convient mieux aux charges moyennes.

Les vérins permettent un mouvement linéaire direct, offrent une bonne force (surtout en version hydraulique), et s'adaptent à de nombreuses applications. Les vérins pneumatiques sont rapides et économiques, tandis que les électriques sont précis et faciles à automatiser. Cependant,les vérins hydrauliques et pneumatiques nécessitent des équipements annexes (pompes, compresseurs), peuvent fuir, et manquent parfois de précision. Les vérins électriques, quant à eux, sont moins puissants et plus coûteux pour des usages intensifs.

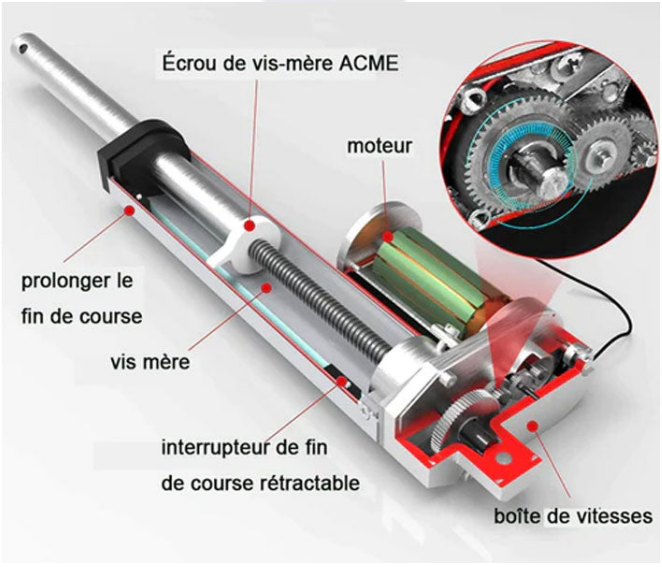



FIGURE 3.11 – Image d'un vérin électrique

Chapitre 4

# Etude des Capteurs

## 4.1. Généralité sur les capteurs

Les capteurs sont des dispositifs fondamentaux dans les systèmes électroniques et automatisés. Leur rôle principal est de percevoir une grandeur physique ou chimique (comme la température, la lumière, la pression, la distance, le gaz, etc.) et de la transformer en un signal électrique exploitable par une machine (microcontrôleur, automate, ordinateur, etc.).
Grâce aux capteurs, un système devient capable d'interagir avec son environnement de manière autonome. On les retrouve partout : dans les smartphones, les voitures, les maisons intelligentes, les appareils médicaux, les robots, et bien plus encore.

Ce chapitre est consacré, dans un premier temps, à une présentation générale des capteurs à travers leur définition, leurs principales caractéristiques et leur classification selon différents critères. Nous aborderons ensuite les différents types de capteurs, en mettant un accent particulier sur les capteurs d'inclinaison, essentiels pour notre application.

### 4.1.1. Définition et caractéristique générale

Un capteur est un dispositif capable de détecter une grandeur physique, chimique ou biologique (telle que la température, la pression, la lumière, l'humidité, etc.) et de la convertir en un signal exploitable, généralement électrique, en vue d'un traitement, d'un affichage ou d'une transmission. Il constitue l'interface entre le monde réel et un système de traitement automatisé.

- **La mesurande :** c'est la grandeur physique, chimique ou biologique à mesurer ;
  Exemple : température, pression, etc.
- **Détecteur primaire ou élément sensible :** c'est la partie du capteur qui interagit directement avec la mesurande. C'est lui qui subit une modification physique (résistance, tension, charge, etc) lorsqu'il est exposé à la grandeur mesurée ;
- **Transducteur ou convertisseur de signal :** il convertit la variation physique détectée par l'élément sensible en un signal électrique (tension, courant, fréquence).
  NB : parfois la partie sensible et le transducteur ne forment qu'un seul bloc ;

- **Conditionneur de signal :** il adapte le signal électrique pour le rendre exploitable par le système de traitement (amplification, filtrage, conversion analogique-numérique ;
- **L'interface de sortie :** la partie du dispositif qui transmet le signal mesuré, généralement sous forme électrique, vers un système de traitement (comme un microcontrôleur, un afficheur ou un automate). Elle peut fournir un signal analogique, numérique, logique ou encore un signal codé, selon le type de capteur et l'application visée [13].

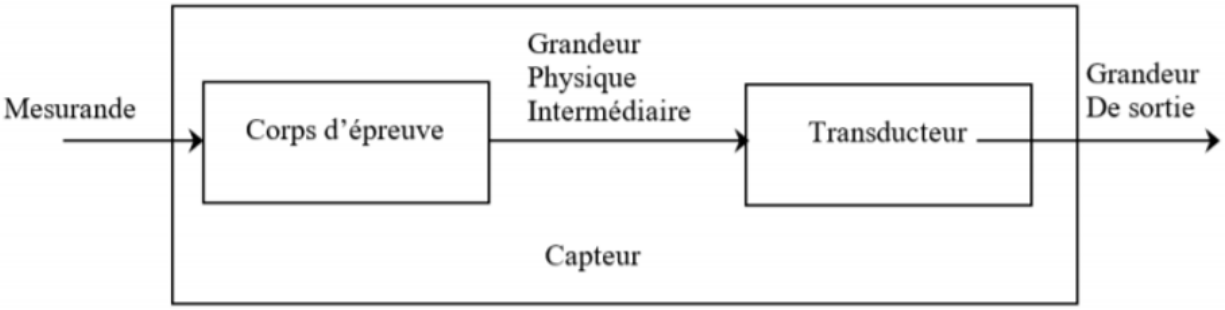



FIGURE 4.1 – Structure d'un capteur

### 4.1.2. Classification des capteurs

Les capteurs peuvent être classés selon plusieurs critères, notamment en fonction de la grandeur à mesurer (ou mesurande), de la nature du signal de sortie, ou encore du mode de fonctionnement.

#### Classification selon la mesurande

Ce critère classe les capteurs en fonction du type de grandeur physique, chimique ou biologique qu'ils détectent.

| Types de capteurs | Grandeur physique détectée | Exemples |
|---|---|---|
| Capteurs de température | Température ambiante, corporelle | LM35, DHT11, thermistance |
| Capteurs de lumière | intensité lumineuse | LDR, photodiode |
| Capteurs de distance | distance, proximité | HC-SR04 |
| Capteurs de pression | pression atmosphérique ou mécanique | BMP180, MPX5010 |
| Capteurs de gaz | Concentration de gaz | MQ-2, MQ-135 |
| Capteurs d'humidité | humidité relative | DHT11, DHT22 |
| Capteurs de son | niveau sonore | microphone, capteur piézoélectrique |
| Capteurs de mouvement | accélération, inclinaison, vibration | MPU6050, ADX345 |
| Capteurs de position | position, déplacement angulaire | potentiomètre, encodeur rotatif |

TABLE 4.1 – Classification des capteurs selon la mesurande

## Classification selon la valeur de sortie

Les capteurs peuvent également être classés en fonction de la nature de leur signal de sortie : analogique, numérique ou binaire (tout ou rien – TOR). Les capteurs analogiques produisent un signal continu, proportionnel à la grandeur mesurée. Les capteurs numériques délivrent des valeurs discrètes, souvent sous forme codée. Quant aux capteurs binaires (ou TOR), ils n'ont que deux états possibles, généralement interprétés comme "présence/absence" ou "marche/arrêt".

| **Type** | **Description** | **Exemples** |
|---|---|---|
| Capteurs analogiques | Fournissent un signal continu proportionnel à la grandeur mesurée | LM35, LDR, potentiomètre |
| Capteurs numériques | Donnent une valeur logique 0 ou 1 ou une donnée numérique codée | DHT11, MPU6050 |
| Les capteurs logiques ou Tout Ou Rien (TOR) | Ils portent le nom de détecteurs.Type de signal de sortie 0V ou 5V | Bouton poussoir, capteur inductif de proximité, capteur infrarouge |

TABLE 4.2 – Classification des capteurs selon le type de signal de sortie

## Classification selon le mode de fonctionnement

Ce critère tient compte de la manière dont le capteur interagit avec la grandeur à mesurer. Les capteurs peuvent être classés selon leur mode de fonctionnement en deux grandes catégories :

- Capteurs passifs : ils ne nécessitent pas d'alimentation électrique externe. Ils produisent un

signal directement en réponse à la grandeur physique mesurée ;

- Capteurs actifs : ils nécessitent une source d'énergie externe pour fonctionner. Cette alimentation permet au capteur de générer ou de conditionner un signal de sortie.

| Type de capteur | Méthode | Principe | Exemples |
|---|---|---|---|
| Passif | Capteurs résistifs | Variation de la résistance électrique sous l'effet d'une grandeur physique | Thermistance (NTC/PTC), LDR |
| Passif | Capteurs capacitifs | Variation de la capacité électrique en fonction de la distance ou du matériau | Capteur de niveau, détecteur de proximité capacitif |
| Actif | Capteurs inductifs | Variation de l'inductance ou détection d'un objet métallique par champ magnétique | Capteur de proximité inductif |
| Actif | Capteurs optiques | Utilisation d'un faisceau lumineux (réflexion, interruption) | Photodiode (en mode actif), capteur infrarouge, barrière optique |
| Passif | Capteurs piézoélectriques | Génèrent une tension lorsqu'ils sont soumis à une contrainte mécanique | Capteur de vibration, capteur de choc, piezo |
| Actif | Capteurs magnétiques | Détection d'un champ magnétique, souvent via effet Hall | Capteur à effet Hall, capteur de position magnétique |

TABLE 4.3 – Classification des capteurs selon le principe de fonctionnement

### 4.1.3. Principe de fonctionnement des capteurs

Un capteur fonctionne en transformant une grandeur physique (telle que la température, la pression, le mouvement, etc.) en une grandeur électrique, afin de faciliter sa transmission et son traitement. Cette conversion repose sur divers principes physiques, et le signal obtenu peut être de nature analogique ou numérique, selon le type de capteur utilisé.

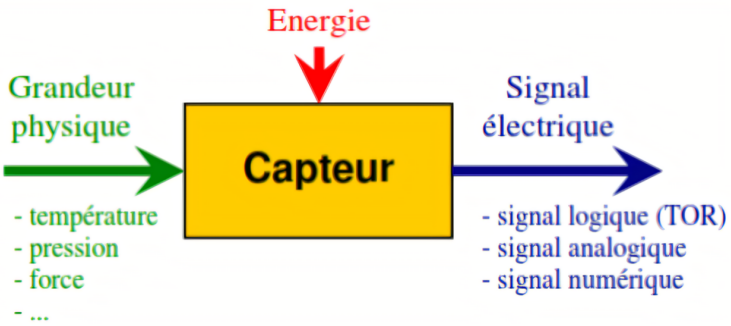



FIGURE 4.2 – Fonctionnement d'un capteur

## 4.2. Les capteurs d'inclinaison

### 4.2.1. Définition

Un capteur d'inclinaison, ou inclinomètre, est un dispositif électronique ou mécanique conçu pour mesurer l'angle d'inclinaison d'un objet par rapport à une référence horizontale ou verticale. Il permet de déterminer l'orientation d'un corps fixe ou en mouvement dans un ou plusieurs plans (généralement un ou deux axes). Ce type de capteur est utilisé pour détecter des changements d'angle, assurer l'alignement, ou surveiller la stabilité et la position angulaire d'un système.

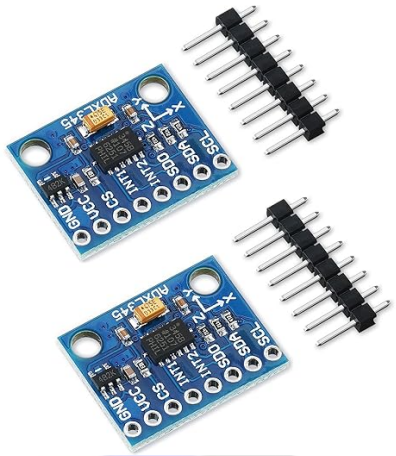



FIGURE 4.3 – Le capteur d'inclinaison ADXL345

### 4.2.2. Principe de fonctionnement

Un capteur d'inclinaison est conçu pour mesurer l'angle d'un objet par rapport à une référence horizontale ou verticale, en utilisant la gravité terrestre comme base de mesure. Le principe repose sur la détection des variations d'orientation de l'objet dans l'espace, en réponse à une inclinaison. Dans le cas des capteurs d'inclinaison de type **MEMS** (*Micro-Electro-Mechanical Systems*), le fonctionnement repose sur la présence d'une *micro-structure mobile* intégrée dans une puce en silicium. Cette structure, souvent constituée d'une masse suspendue (ou masse d'inertie), est libre de se déplacer légèrement sous l'effet de la gravité lorsque le capteur est incliné.
Ce déplacement entraîne une modification d'une grandeur physique mesurable, généralement :

- une **variation de capacité** entre des électrodes fixes et mobiles (dans les capteurs capacitifs) ;
- ou une **variation de résistance** mécanique (dans les capteurs piézorésistifs).

Ces variations sont ensuite détectées et traitées par un circuit électronique intégré au capteur, qui les convertit en un *signal proportionnel à l'angle d'inclinaison.*

Le signal de sortie peut être :

- **analogique**, sous forme d'une tension variable proportionnelle à l'angle mesuré ;
- ou **numérique**, exprimé directement en degrés via un protocole de communication (comme $I^2C$ ou SPI).

Ce principe permet de mesurer l'inclinaison selon un ou deux axes, avec une bonne précision, une grande compacité et une faible consommation d'énergie.
Les inclinomètres MEMS sont ainsi largement utilisés dans les systèmes embarqués, l'alignement de structures, les dispositifs de stabilisation ou encore les applications robotiques [14].

### 4.2.3. Les types de capteurs d'inclinaison

Les capteurs d'inclinaison se distinguent principalement selon deux critères : la technologie de détection utilisée (électrolytique, capacitive, MEMS, etc.) et le nombre d'axes de mesure (monodirectionnel ou bidirectionnel). Le choix du type dépend des exigences en termes de précision, de vitesse de réponse, d'environnement d'utilisation et de dimensions mécaniques.

| Type de capteur | Principe de fonctionnement | Technologie | Direction de mesure | Applications courantes |
|---|---|---|---|---|
| Électrolytique | Variation de conductivité entre électrodes selon le niveau d'un liquide conducteur | Analogique | Mono ou bi-axial | Surveillance de structures, inclinomètres industriels |
| À bulle | Position d'une bulle dans un liquide selon l'inclinaison | Mécanique | Monodirectionnel | Outils de nivellement, dispositifs manuels |
| À pendule | Mouvement d'un pendule détecté mécaniquement (optique ou potentiométrique) | Mécanique ou Électromécanique | Mono ou bi-axial | Contrôle de stabilité, sécurité de machines |
| Piézoélectrique | Génération d'une tension via la contrainte sur un cristal sous l'effet de l'inclinaison | Électrique | Monodirectionnel | Détection de vibrations rapides et chocs |
| Capacitif | Variation de capacité entre électrodes due au déplacement d'une masse interne | Électronique | Mono ou bi-axial | Stabilisation, instrumentation mobile |
| MEMS | Déplacement d'une micro-masse détecté par effet capacitif ou piézorésistif intégré à une puce | Électronique intégrée (MEMS) | Mono ou bi-axial | Robotique, drones, smartphones, dispositifs embarqués |

TABLE 4.4 – Types de capteurs d'inclinaison

### 4.2.4. Le capteur d'inclinaison MPU6050

Le MPU6050 est un capteur inertiel de type IMU 6 axes fabriqué par InvenSense. Il combine dans un seul boîtier un accéléromètre triaxial et un gyroscope triaxial, avec un processeur de mouvement numérique (DMP) intégré, ce qui permet d'effectuer des calculs embarqués de mouvements complexes.
L'accéléromètre donne une idée globale de l'inclinaison par rapport à la verticale (utile quand l'objet est immobile).
Le gyroscope permet de suivre les mouvements rapides et d'estimer les rotations avec précision.
Ensemble, ils permettent un calcul fiable, stable et réactif de l'angle réel (grâce à une méthode appelée fusion de capteurs, par exemple le filtre complémentaire ou le filtre de Kalman).

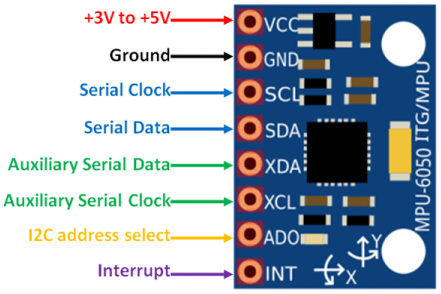



FIGURE 4.4 – Le capteur MPU6050

Le tableau ci dessous renseigne sur les caractéristiques du MPU6050 [5].

| Caractéristique | Valeur / Plage | Remarque |
|---|---|---|
| Interface de communication | $I^2C$ (jusqu'à 400 kHz) | Compatible avec Arduino |
| Tension d'alimentation | 2,375- 3,46 V | Alimentation typique : 3,3V |
| Accéléromètre intégré | 3 axes, 16 bits | Plages : ±2, ±4, ±8, ±16g |
| Sensibilité accéléromètre | 16384 LSB/g (±2g) | Haute résolution |
| Consommation accéléromètre | Environ 500µA | Basse consommation |
| Gyroscope intégré | 3 axes, 16 bits | Plages : ±250, ±500, ±1000, ±2000°/s |
| Sensibilité gyroscope | 131 LSB/(°/s) (±250°/s) | Bonne linéarité |
| Consommation gyroscope | Environ 3,6 mA | Actif en continu |
| DMP intégré | Oui | Permet filtrage et fusion capteurs |
| Température de fonctionnement | -40 C à +85°C | Avec capteur thermique interne |
| Boîtier | QFN 4×4 mm, 24 broches | Très compact |
| Fréquence d'échantillonnage | Jusqu'à 1 kHz | Configurable via registre |
| Filtre passe-bas numérique | Oui, programmable | Pour gyro, accéléro et température |

TABLE 4.5 – Caractéristiques techniques essentielles du capteur MPU-6050

# Synthèse I

Cette première partie a permis d'établir les bases nécessaires à la conception du système de calage automatique de l'angle d'un plan. L'étude de l'architecture Arduino et de son environnement de développement a mis en évidence ses capacités de contrôle et d'interaction avec les composants.

Le rappel sur les composants électroniques et électromécaniques a renforcé la compréhension des éléments essentiels, notamment moteurs, relais et transistors. Une attention particulière a été portée aux capteurs, avec un focus sur le MPU6050, son fonctionnement et son intégration via Arduino.

Ainsi, l'ensemble des connaissances théoriques acquises constituent une base solide pour aborder la simulation et la réalisation concrète du système.

# Deuxième partie

# Conception et Simulation du Système

Chapitre 5

# Description fonctionnelle du système

## 5.1. Objectif du système

Le projet a pour objectif la mise en œuvre d'un dispositif capable de mesurer un angle d'orientation, de l'afficher en temps réel, puis de générer un signal de commande permettant d'ajuster automatiquement l'angle d'un plan dans l'espace, afin de l'aligner sur une valeur cible définie manuellement.
Dans le cadre de la simulation, l'étude est restreinte à la correction de l'angle dans une seule dimension (par exemple : l'axe de site), ce qui permet de valider le principe de fonctionnement de manière simplifiée.

## 5.2. Logique de fonctionnement du système

La simulation du système a été réalisée dans l'environnement Proteus, avec les composants suivants : un potentiomètre, une carte Arduino Uno, un écran LCD I2C et un servomoteur.
Le potentiomètre, connecté à la broche A1 de l'Arduino, génère une valeur analogique comprise entre 0 et 1023, laquelle est convertie en une valeur d'angle cible en site, allant de 0 à 90 degrés. Cette valeur représente l'orientation souhaitée du plan.
Étant donné que Proteus ne permet pas de représenter physiquement un plan incliné, la valeur de l'angle mesuré est simulée directement dans le code Arduino injecté dans la carte. Ainsi, le système repose sur une comparaison logicielle entre l'angle mesuré (simulé) et l'angle cible (réglé via le potentiomètre).
En cas de différence entre ces deux valeurs, le microcontrôleur Arduino Uno commande un servomoteur, de manière à corriger l'orientation du plan simulé vers l'angle cible. L'ajustement s'effectue par pas successifs jusqu'à atteindre l'alignement souhaité.
L'ensemble du processus est visualisé en temps réel grâce à un écran LCD I2C, qui affiche simultanément les deux valeurs d'angle : celle mesurée (ou simulée) et celle définie par l'utilisateur.
Ce principe de fonctionnement pose les bases de la mise en œuvre du système. Afin de concrétiser cette logique, un choix judicieux des composants a été effectué en tenant compte des contraintes de simulation, de précision, de disponibilité et de simplicité d'intégration.

La section suivante présente donc les éléments matériels sélectionnés, ainsi que leurs rôles respectifs dans l'architecture du système.

## 5.3. Choix des composants

Pour la simulation du système, plusieurs composants électroniques ont été sélectionnés en fonction de leur disponibilité dans l'environnement de simulation Proteus, de leur compatibilité avec Arduino, et de leur adéquation avec les objectifs du projet. Chaque composant contribue à une fonction précise dans la chaîne de traitement, allant de la saisie de l'angle cible à la correction automatique via un actionneur.

Le tableau ci-dessous présente les composants utilisés ainsi que leur rôle spécifique dans la simulation.

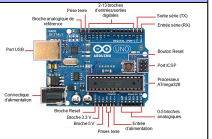
| Composant | Rôle dans la simulation | Illustration |
|---|---|---|
| **Arduino Uno** | Cœur du système. Il lit la valeur du potentiomètre (angle cible), génère une valeur simulée de l'angle mesuré, effectue la comparaison entre les deux, commande le servomoteur et met à jour l'affichage sur l'écran LCD. |  |
| **Potentiomètre** | Sert à définir manuellement l'angle cible. Connecté à l'entrée analogique A1 de l'Arduino, il fournit une valeur comprise entre 0 et 1023, convertie en un angle de 0° à 90°. |  |
| **Servomoteur** | Représente l'actionneur chargé de corriger l'orientation du plan. Il est contrôlé par un signal PWM généré par l'Arduino en fonction de l'écart entre l'angle cible et l'angle mesuré. |  |
| **Écran LCD I2C (16x2)** | Permet l'affichage en temps réel des valeurs d'angle cible et d'angle mesuré. L'interface I2C réduit le nombre de broches nécessaires sur l'Arduino. |  |
| **Alimentation 5V** | Fournit l'énergie nécessaire au fonctionnement de l'ensemble des composants. Dans la simulation, l'alimentation est représentée par l'alimentation interne de Proteus. | |

TABLE 5.1 – Composants utilisés et leur rôle dans la simulation

Notons qu'en lieu et place du servomoteur, il est également possible d'utiliser un moteur pas à pas piloté par un module ULN2003, lequel permet de faire tourner automatiquement l'axe afin d'ajuster l'angle mesuré à l'angle cible défini par le potentiomètre.

## 5.4. Schéma du montage sur Proteus

Afin de valider le fonctionnement théorique du système, un modèle du montage a été conçu et simulé à l'aide du logiciel Proteus, permettant ainsi de visualiser le comportement des composants et d'anticiper les résultats avant la mise en œuvre physique. Le schéma se présente comme suit :

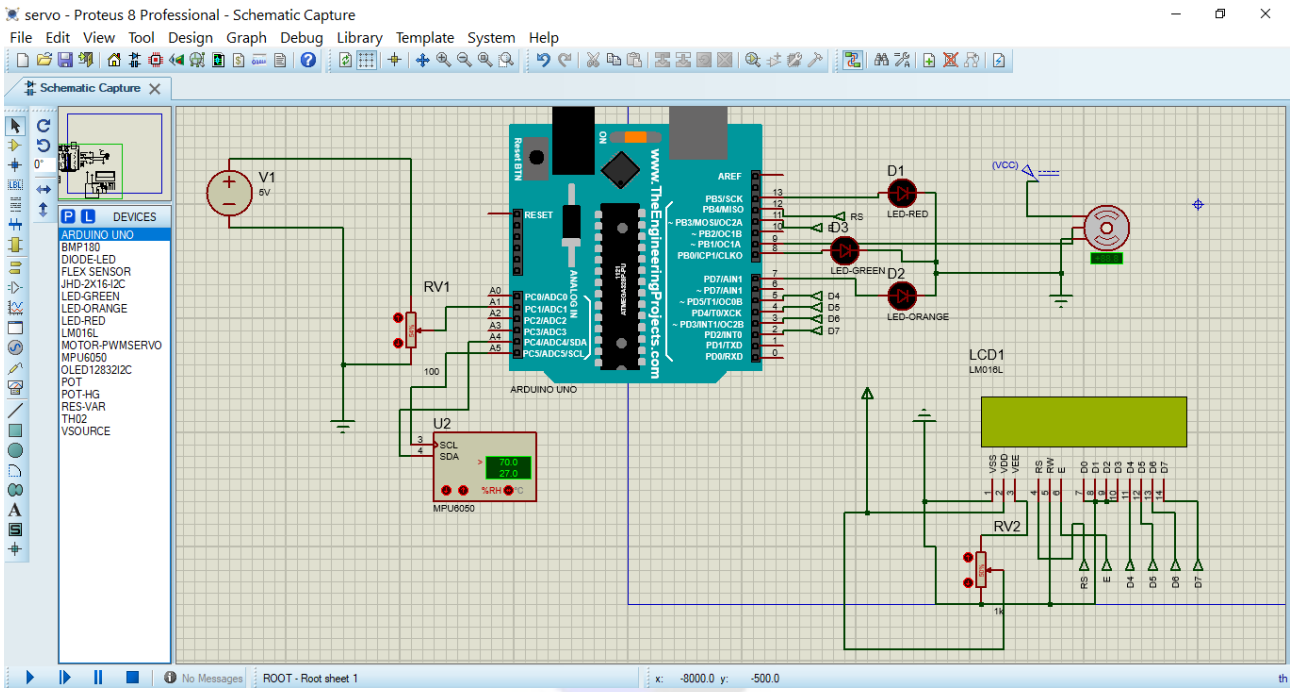



FIGURE 5.1 – Schéma de la simulation

En réalité, le capteur d'inclinaison MPU6050 n'étant pas disponible dans la bibliothèque de Proteus, sa présence dans le montage simulé a été simplement symbolisée afin de rapprocher autant que possible la simulation du montage réel. Par ailleurs, des LED ont été intégrées au circuit dans un objectif d'illustration visuelle : la LED rouge s'allume lorsque l'écart entre l'angle mesuré et l'angle cible est significatif, la LED orange indique un écart modéré, tandis que la LED verte s'allume pour signaler que l'angle mesuré est en parfaite concordance avec l'angle cible.

## 5.5. Branchement des composants

### 5.5.1. Connexion du potentiomètre à la carte arduino uno

Le potentiomètre est relié à l'Arduino Uno via trois connexions principales : l'alimentation positive du potentiomètre est branchée sur la broche 5V, son signal de sortie est connecté à la broche analogique A0, et sa masse est reliée à la broche GND. Cette configuration permet d'alimenter le

potentiomètre, de lire sa valeur variable et de l'intégrer dans les programmes de la carte Arduino.

| Broche Potentiomètre | Broche Arduino Uno | Fonction |
|---|---|---|
| Alimentation (+) | 5V | Alimentation 5V |
| Signal | A0 | Entrée analogique |
| Masse (-) | GND | Masse |

TABLE 5.2 – Branchements entre le potentiomètre et la carte Arduino Uno

### 5.5.2. Connexion du servomoteur à la carte arduino uno

Le servomoteur est relié à l'Arduino Uno par trois connexions : l'alimentation (5V) pour fournir la tension nécessaire, la masse (GND) pour une référence commune, et la broche numérique 3 pour le signal PWM permettant le contrôle précis de l'angle. Cette configuration assure à la fois l'alimentation et le pilotage fiable du servomoteur.

| Broche Servomoteur | Broche Arduino Uno | Fonction |
|---|---|---|
| Alimentation (+) | 5V | Alimentation 5V |
| Signal | PD3/OC2B (pin 3) | Signal de contrôle PWM |
| Masse (-) | GND | Masse |

TABLE 5.3 – Branchements entre le servomoteur et la carte Arduino Uno

### 5.5.3. Connexion de l'écran LCD à la carte arduino uno

Le branchement de l'écran LCD à la carte Arduino Uno se fait à l'aide de plusieurs connexions. L'alimentation de l'écran LCD est reliée aux broches 5V et GND de l'Arduino, afin de fournir la tension nécessaire au fonctionnement de l'écran. Ensuite, les broches de contrôle de l'écran LCD (RS, E, D4, D5, D6, D7) sont connectées à des broches numériques spécifiques de la carte Arduino. Cela permet à l'Arduino de communiquer avec l'écran LCD et d'afficher les informations souhaitées.

| Broche LCD | Broche Arduino Uno | Fonction |
|---|---|---|
| VCC | 5V | Alimentation 5V |
| GND | GND | Masse |
| RS | PD7 (pin 7) | Sélection du registre |
| E | PD6 (pin 6) | Validation des données |
| D4 | PD5 (pin 5) | Données (bit 4) |
| D5 | PD4 (pin 4) | Données (bit 5) |
| D6 | PD3 (pin 3) | Données (bit 6) |
| D7 | PD2 (pin 2) | Données (bit 7) |

TABLE 5.4 – Branchements entre l'écran LCD et la carte Arduino Uno

## 5.6. Analyse du Code Arduino pour la Simulation sous Proteus

Le programme suivant, développé dans l'environnement Arduino IDE, a été téléversé dans la carte Arduino UNO pour simuler le comportement du système sous Proteus. Il permet de comparer en temps réel un angle mesuré avec un angle cible (défini par un potentiomètre). Le servomoteur effectue alors un ajustement progressif afin d'atteindre la valeur de l'angle cible renseignée. L'état du système est affiché sur un écran LCD 16x2, tandis que des LEDs signalent les différents niveaux d'écart. Ce programme illustre la logique de régulation mise en œuvre et constitue le cœur du fonctionnement de notre dispositif.

```
#include <Servo.h> // Bibliotheque pour le servomoteur
#include <Wire.h> // Communication I2C (non utilisee ici mais souvent incluse)
#include <LiquidCrystal.h>   // Bibliotheque pour l'ecran LCD

#define SERVO_PIN 9     // Broche de controle du servomoteur

Servo myServo;    // Objet pour controler le servomoteur
LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // Broches connectees au LCD

int currentAngle;   // Angle actuel simule

void setup() {
 // Initialisation des LEDs
  pinMode(13, OUTPUT);    // LED rouge (ecart important)
  pinMode(8, OUTPUT);     // LED verte (pas d'erreur)
  pinMode(7, OUTPUT);     // LED orange (ecart modere)

  Serial.begin(9600);
  randomSeed(analogRead(0));     // Initialisation aleatoire de l'angle actuel
  currentAngle = random(0, 180);

  myServo.attach(SERVO_PIN);     // Lier le servomoteur a sa broche
  lcd.begin(16, 2);              // Initialisation du LCD 16x2

 // Sequence de bienvenue
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("   WELCOME   ");
  delay(600);
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("    CALAGE");
  lcd.setCursor(0, 1); lcd.print(" AUTOMATIQUE ");
  delay(600);
  lcd.clear();
```

```
    lcd.setCursor(0, 0); lcd.print("ENTRER L'ANGLE");
    lcd.setCursor(0, 1); lcd.print("   CIBLE");
    delay(2000);

  // Lecture initiale de l'angle cible
    int potValue = analogRead(A1);                          // Lecture du
        potentiometre
    int targetAngle = map(potValue, 0, 1023, 0, 180);       // Conversion
        en angle
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("ANGLE CIBLE: ");
    lcd.setCursor(13, 0); lcd.print(targetAngle);
    delay(3000);
  }

  void loop() {
   int vitesse = 2;                                         // Vitesse de
       correction
   int potValue = analogRead(A1);                           // Lecture du
       potentiometre cible
   int targetAngle = map(potValue, 0, 1023, 0, 180);        // Conversion
       en angle
   int angleError = currentAngle - targetAngle;

   // Ajustement de l'angle mesure vers l'angle cible
   if (currentAngle < targetAngle) {
    currentAngle += vitesse;
    if (currentAngle > targetAngle) currentAngle = targetAngle;
   } else if (currentAngle > targetAngle) {
    currentAngle -= vitesse;
    if (currentAngle < targetAngle) currentAngle = targetAngle;
   }

   // Eteindre toutes les LEDs
   digitalWrite(13, LOW);
   digitalWrite(8, LOW);
   digitalWrite(7, LOW);

   lcd.clear();

   // Si un ecart est detecte
   if (abs(angleError) > 1) {
    myServo.write(currentAngle); // Mise a jour du servo

    // Signalisation selon la gravite de l'ecart
    if (abs(angleError) > 20) {
     digitalWrite(13, HIGH);  // Ecart important -> LED rouge
    }
   else {
```

```
78      digitalWrite(7, HIGH);        // Ecart modere -> LED orange
79    }
80
81    // Affichage des informations sur le LCD
82    lcd.setCursor(0, 0); lcd.print("Actuel: "); lcd.print(currentAngle);
83    lcd.setCursor(0, 1); lcd.print("Cible: "); lcd.print(targetAngle);
84    delay(600);
85
86    lcd.clear();
87    lcd.setCursor(0, 0); lcd.print("Ecart: "); lcd.print(angleError);
88    delay(500);
89
90    lcd.clear();
91    lcd.setCursor(0, 0); lcd.print("CORRECTION");
92    lcd.setCursor(0, 1); lcd.print("  EN COURS ");
93    delay(700);
94   } else {
95    // Aucun ecart significatif
96    myServo.write(currentAngle);                                  // Maintien de
        la position
97    digitalWrite(8, HIGH);                                        // LED verte
        allumee
98    lcd.setCursor(0, 0); lcd.print("AUCUNE ERREUR");
99    lcd.setCursor(0, 1); lcd.print("  DETECTEE ");
100   delay(600);
101  }
102 }
```

### 5.6.1. Explication des mots-clés du code

Afin de faciliter la compréhension du programme implémenté, les mots-clés essentiels employés dans le code, ainsi que leur rôle spécifique, sont présentés dans le tableau ci-après. Cette synthèse permet de mieux cerner la structure et la logique du code développé.

| Mot-clé / Fonction | Rôle dans le code |
|---|---|
| `#include <Servo.h>` | Importe la bibliothèque pour contrôler un servomoteur via PWM. |
| `#include <Wire.h>` | Importe la bibliothèque pour la communication I2C (non utilisée ici). |
| `#include <LiquidCrystal.h>` | Permet d'utiliser un écran LCD 16x2. |
| `#define SERVO_PIN 9` | Définit une constante représentant la broche du servomoteur. |
| `Servo myServo` | Crée un objet permettant de manipuler le servomoteur. |
| `LiquidCrystal lcd(...)` | Crée un objet LCD connecté aux broches spécifiées. |
| `void setup()` | Fonction exécutée une seule fois au démarrage du programme. |
| `void loop()` | Fonction principale exécutée en boucle infinie. |
| `pinMode(pin, mode)` | Configure une broche comme entrée ou sortie (ex : LEDs). |
| `digitalWrite(pin, value)` | Met une broche à l'état haut (`HIGH`) ou bas (`LOW`). |
| `analogRead(pin)` | Lit une tension analogique (0–5V) et retourne une valeur entre 0 et 1023. |
| `map(val, in1, in2, out1, out2)` | Convertit une valeur d'une plage à une autre (ex : 0–1023 $\rightarrow$ 0–180). |
| `delay(ms)` | Met en pause l'exécution du programme pendant un certain nombre de millisecondes. |
| `lcd.begin(16, 2)` | Initialise l'écran LCD 16 colonnes × 2 lignes. |
| `lcd.setCursor(col, row)` | Positionne le curseur à un emplacement donné sur le LCD. |
| `lcd.print(...)` | Affiche un texte ou une valeur sur l'écran LCD. |
| `lcd.clear()` | Efface le contenu de l'écran LCD. |
| `randomSeed(val)` | Initialise le générateur de nombres pseudo-aléatoires. |
| `random(min, max)` | Génère un nombre aléatoire compris dans une plage donnée. |
| `myServo.attach(pin)` | Lie le servomoteur à une broche PWM. |
| `myServo.write(angle)` | Positionne le servomoteur à un angle compris entre 0° et 180°. |
| `Serial.begin(9600)` | Démarre la communication série à 9600 bauds. |
| `int` | Type de variable entière utilisée pour stocker des angles, erreurs, etc. |
| `currentAngle` | Stocke l'angle actuel simulé du système. |
| `targetAngle` | Stocke l'angle cible défini par le potentiomètre. |
| `angleError` | Différence entre l'angle actuel et l'angle cible. |
| `abs(valeur)` | Calcule la valeur absolue d'un nombre (utile pour comparer les écarts). |
| `if / else if / else` | Structures conditionnelles pour exécuter des actions selon des cas logiques. |

TABLE 5.5 – Résumé des mots-clés et fonctions utilisés dans le code Arduino

# Synthèse II

Au terme de cette phase consacrée au montage et à la simulation, le fonctionnement théorique du système de calage automatique de l'angle d'inclinaison d'un plan a pu être validé à travers l'environnement de simulation. Cette étape s'est révélée essentielle pour anticiper certaines contraintes techniques et affiner les paramètres du montage avant toute implémentation réelle. Fort de ces résultats encourageants, le projet peut désormais évoluer vers la prochaine étape : la réalisation pratique du système, qui fera l'objet de la partie suivante.

# Troisième partie

# Réalisation Pratique du Dispositif

Chapitre 6

# Implémentation réelle du circuit

## 6.1. Choix et acquisition des composants

La réalisation pratique du système de calage automatique de l'angle d'inclinaison d'un plan nécessite l'utilisation de divers composants électroniques et électromécaniques, soigneusement sélectionnés en fonction des contraintes fonctionnelles du projet. Cette section présente les éléments matériels retenus, la justification de leur choix ainsi que les modalités de leur acquisition.

### 6.1.1. Liste du matériel utilisé

Le tableau ci-dessous regroupe les principaux composants nécessaires à la mise en œuvre du dispositif :

| Désignation | Référence ou modèle | Quantité |
|---|---|---|
| Carte microcontrôleur | ESP32 WROOM | 1 |
| Capteur d'angle/inclinaison | MPU6050 (accéléromètre + gyroscope) | 1 |
| Potentiomètre | 10 kΩ | 1 |
| Servomoteur | SG90 | 1 |
| Moteur pas à pas | 28BYJ-48 | 1 |
| Module de commande moteur | ULN2003 | 1 |
| Afficheur LCD | LCD 20x4 avec interface I2C | 1 |
| Résistances | 220 Ω, 10 kΩ | Divers |
| LED de signalisation | Rouge, orange, verte | 3 |
| Alimentation externe | 5V via USB ou adaptateur | 1 |
| Câbles de connexion | Dupont (M/M, M/F) | Divers |
| Plaque d'expérimentation | Breadboard | 1 |
| Support mécanique | Contreplaqué découpé | 1 |

TABLE 6.1 – Liste des composants utilisés pour le montage réel

### 6.1.2. Justification du choix des composants

Les motivations et critères ayant conduit à la sélection des éléments matériels retenus pour la réalisation pratique du dispositif s'articulent comme suit :

- **ESP32-WROOM** : Ce module microcontrôleur hautement performant intègre un processeur dual-core, une connectivité Wi-Fi et Bluetooth Low Energy (BLE) natives, ainsi qu'une grande variété de broches d'entrées/sorties. Son architecture puissante et polyvalente permet un traitement rapide des données issues des capteurs et un contrôle précis des actionneurs, tout en offrant une consommation énergétique optimisée. L'ESP32-WROOM est ainsi parfaitement adapté à la gestion simultanée des capteurs d'inclinaison, des moteurs et des interfaces utilisateur dans un environnement embarqué ;
- **MPU6050 (accéléromètre + gyroscope) :** Ce capteur inertiel 6 axes fournit une mesure fiable de l'orientation spatiale par la fusion des données d'accélération et de rotation. Son intégration avec l'ESP32 via un protocole I2C simplifie la communication et permet d'obtenir un angle d'inclinaison précis et en temps réel, élément essentiel au bon fonctionnement du système de calage automatique ;
- **Potentiomètre 10 kΩ :** Constitue un moyen simple et efficace pour l'utilisateur de fixer manuellement l'angle de consigne. Sa linéarité et sa compatibilité avec les entrées analogiques de l'ESP32 assurent une lecture stable et précise de la valeur de référence ;
- **Servomoteur SG90 :** Ce servomoteur compact et léger offre une résolution angulaire suffisante pour des ajustements fins. Sa facilité d'intégration, sa disponibilité et son rapport qualité-prix en font un choix pertinent pour le contrôle précis de l'inclinaison dans le système ;
- **Moteur pas à pas 28BYJ-48 avec module ULN2003 :** Le moteur pas à pas permet un contrôle positionnel très précis, grâce à ses pas fixes et répétables. Associé au module ULN2003, qui sert d'interface de puissance, il permet de commander des courants plus élevés que ceux supportés par l'ESP32, tout en protégeant ce dernier des surcharges électriques. Cette configuration garantit un pilotage fiable et sécuritaire du moteur dans les phases d'ajustement ;
- **Afficheur LCD 20x4 avec interface I2C :** L'afficheur permet d'afficher simultanément plusieurs informations telles que l'angle mesuré, l'angle cible, et les états du système. La connexion I2C réduit le nombre de fils nécessaires, simplifiant le câblage et limitant les risques d'erreurs ou d'interférences électriques, tout en facilitant la programmation ;
- **Contreplaqué découpé :** Utilisé comme support mécanique, le contreplaqué assure la rigidité et la stabilité du dispositif. Sa capacité à amortir les vibrations et à supporter le poids des composants mobiles contribue à la précision des mesures et à la durabilité de l'ensemble ;
- **LEDs de signalisation (rouge, orange, verte) :** Ces indicateurs visuels permettent une communication immédiate de l'état du système à l'utilisateur, notamment en signalant la proximité ou l'écart important entre l'angle réel et la consigne, améliorant ainsi l'ergonomie et la réactivité du dispositif.

### 6.1.3. Schéma de cablage réel

Un schéma de câblage a été conçu afin d'illustrer les connexions physiques entre les différents composants, installés sur la breadboard et la carte ESP32. Ce schéma, dérivé du montage simulé, a été adapté pour prendre en considération les contraintes matérielles réelles ainsi que les exigences d'ergonomie du circuit. Il est présenté comme suit :

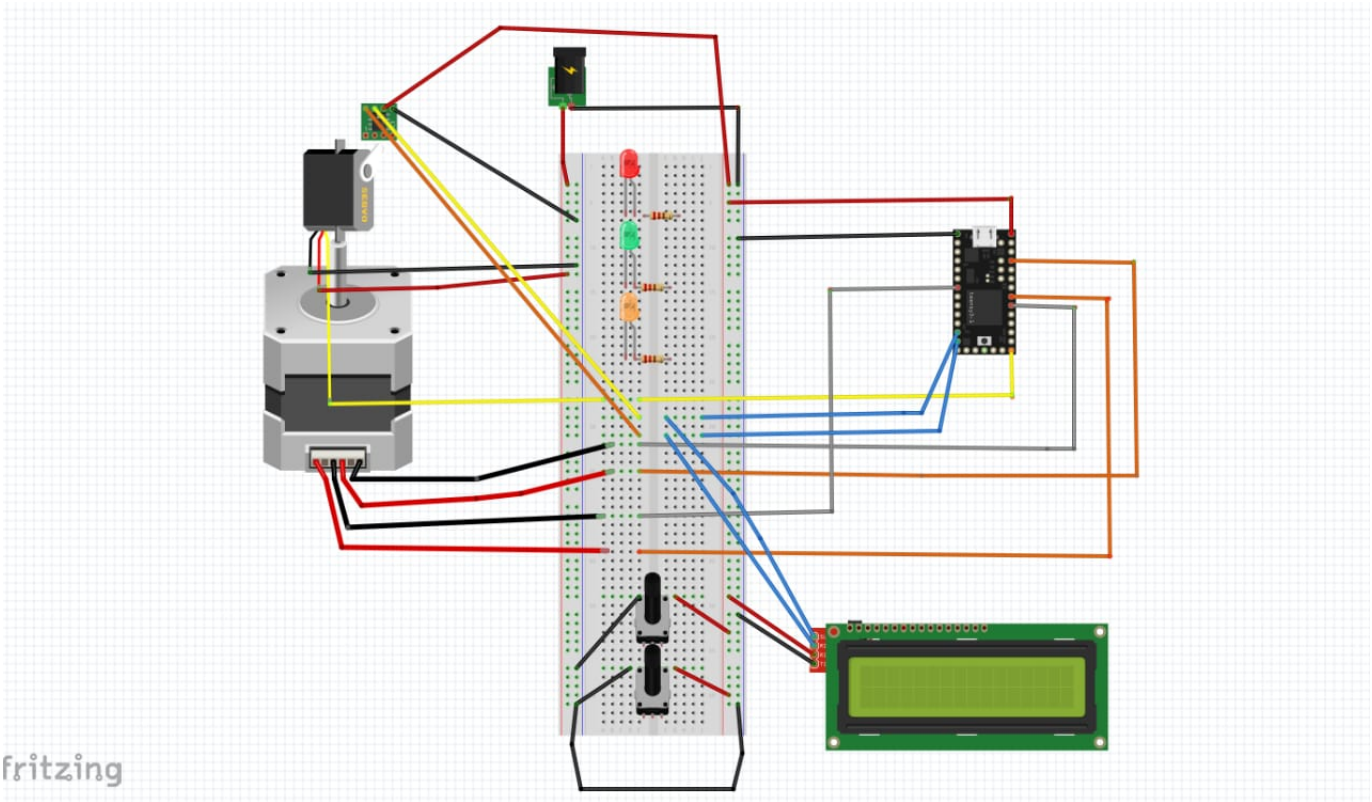



FIGURE 6.1 – Schéma du montage avec le logiciel fritzing

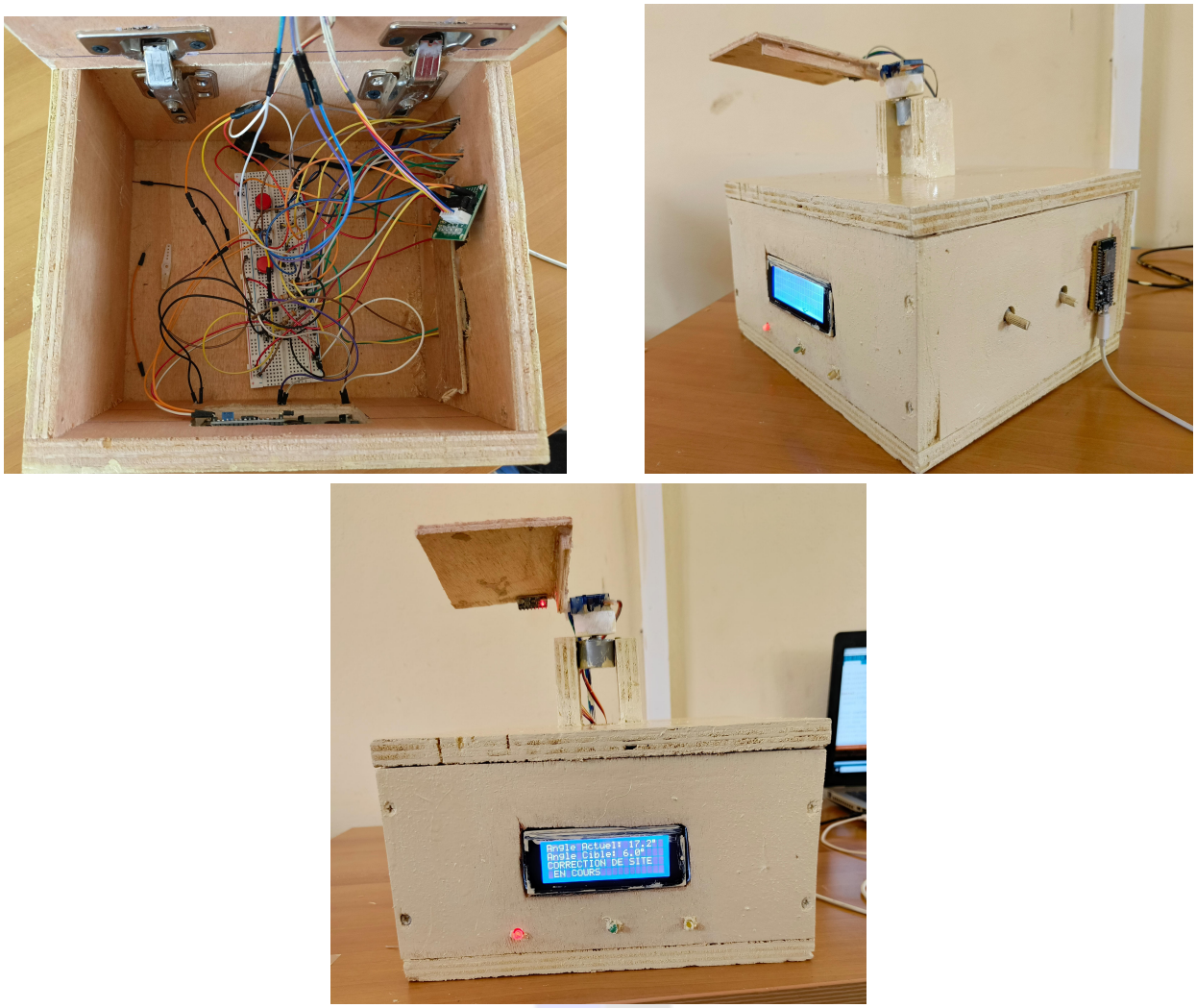



FIGURE 6.2 – Vue du montage finalisé

## 6.2. Code arduino exploité pour le montage

Le programme suivant réalise un système de calage automatique en site (élévation) et en azimut (orientation horizontale) à l'aide d'une ESP32, d'un MPU6050, d'un servomoteur, d'un moteur pas à pas ULN2003, d'un LCD 20x4 I2C et d'une interface Bluetooth. À l'allumage, les modules sont initialisés. L'utilisateur peut définir les angles cibles via des potentiomètres ou, si ceux-ci sont à zéro, les saisir via Bluetooth. Les messages de progression sont affichés sur le LCD et transmis au téléphone.

Le calage du site se fait à l'aide d'un servomoteur, commandé en fonction de la mesure d'angle d'inclinaison issue du MPU6050 (filtrée avec un filtre complémentaire). Tant que l'écart entre l'angle actuel et l'angle cible dépasse une certaine marge, le servo ajuste sa position. Une LED rouge, orange ou verte s'allume pour indiquer le sens ou la réussite de la correction. Une fois le site calé, le système passe à l'azimut, corrigé à l'aide d'un moteur pas à pas. L'angle de rotation est

estimé à partir du gyroscope (axe Z). Le moteur tourne jusqu'à ce que l'angle cible soit atteint, et les informations sont continuellement mises à jour sur l'écran. À la fin, un résumé est affiché, et un message "CALAGE REUSSIE" est envoyé via Bluetooth.

```
#include <ESP32Servo.h>
#include <MPU6050.h>
#include "BluetoothSerial.h"
#include <LiquidCrystal_I2C.h>
#include <Wire.h>
#include <Stepper.h>

// === D clarations des constantes ===
#define SERVO_PIN 13
#define PIN_POT 34
#define PIN_POT1 35
#define PIN_LEDR 4
#define PIN_LEDO 32
#define PIN_LEDV 14
#define LCD_ADDR 0x27
#define MAX_SITE_ANGLE 75
#define MAX_AZIMUT_ANGLE 60
#define ANGLE_MARGIN_SITE 1
#define ANGLE_MARGIN_AZIMUT 2
#define MAX_STEPPER_STEP 2048
#define MIN_STEPPER_STEP 28
#define RPM 10

// === Objets globaux ===
LiquidCrystal_I2C lcd(LCD_ADDR, 20, 4);
Stepper myStepper(MAX_STEPPER_STEP, 18, 23, 19, 5);
BluetoothSerial SerialBT;
Servo myServo;
MPU6050 mpu;

// === Variables de capteur ===
float currentAngle = 0, commande = 0, angleZ = 0;
float axg, ayg, azg, gyg, gzg, accAngle, dt;
const float PAS_CORRECTION = 5;
int16_t ax, ay, az, gx, gy, gz;
unsigned long lastTime = 0;
const float alpha = 0.98;

// === Setup initial ===
void setup() {
  Serial.begin(115200);
  SerialBT.begin("ESP32-calage");

  pinMode(PIN_LEDR, OUTPUT);
  pinMode(PIN_LEDO, OUTPUT);
```

```
  pinMode(PIN_LEDV, OUTPUT);

  myServo.attach(SERVO_PIN);
  myStepper.setSpeed(RPM);
  myServo.write(commande);

  Wire.begin();
  lcd.init();
  lcd.backlight();

  // Initialisation MPU6050
  mpu.initialize();
  if (mpu.testConnection()) {
    lcd.setCursor(0, 0);
    lcd.print("MPU6050 OK");
  } else {
    lcd.setCursor(0, 0);
    lcd.print("Erreur MPU");
    while (1);
  }
  delay(500);

  lcd.setCursor(0, 1);
  lcd.print("CALIBRAGE GYROSCOPE.....");
  mpu.CalibrateGyro();
  delay(1000);

  // Message d'accueil
  lcd.clear();
  lcd.setCursor(0, 2);
  lcd.print("     BIENVENUE");
  delay(2000);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(" CALAGE AUTOMATIQUE");
  lcd.setCursor(0, 1);
  lcd.print(" ANGLE D'INCLINAISON");
  lcd.setCursor(0, 2);
  lcd.print("    PROJET ");
  lcd.setCursor(0, 3);
  lcd.print("  FIN D'ETUDE");
  delay(3000);

  lastTime = millis();
}

// === Lecture d'un angle depuis Bluetooth ===
float lireBluetoothAngle(String message, int minVal, int maxVal) {
  float angle = -1;
```

```
  while (angle < minVal || angle > maxVal) {
    SerialBT.println(message);
    while (!SerialBT.available()) {
      delay(10);
    }
    String input = SerialBT.readStringUntil('\n');
    input.trim();
    if (input.length() == 0) continue;
    angle = input.toFloat();
    if (angle < minVal || angle > maxVal) {
      SerialBT.println("VALEUR INVALIDE. REESSAYEZ.");
    }
  }
  return angle;
}

// === Lecture de l'angle d'inclinaison (Site) ===
float lire_site() {
  for (int i = 0; i < 100; i++) {
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    axg = ax / 16384.0;
    ayg = ay / 16384.0;
    azg = az / 16384.0;
    gyg = gy / 131.0;
    accAngle = atan2(axg, sqrt(ayg * ayg + azg * azg)) * 180.0 / PI;

    unsigned long currentTime = millis();
    dt = (currentTime - lastTime) / 1000.0;
    lastTime = currentTime;

    if (dt > 0)
      currentAngle = alpha * (currentAngle + gyg * dt) + (1 - alpha) *
          accAngle;

    currentAngle = constrain(currentAngle, 0, 180);
  }
  return currentAngle;
}

// === Lecture angle Azimut (Z) ===
float lire_azimut() {
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  gzg = gz / 131.0;
  unsigned long currentTime = millis();
  dt = (currentTime - lastTime) / 1000.0;
  lastTime = currentTime;
  angleZ += gzg * dt;
  angleZ = constrain(angleZ, 0, MAX_AZIMUT_ANGLE);
  return angleZ;
```

```
143 }
144
145 // === Affichage LCD multi-lignes ===
146 void afficherLCD(String ligne1, String ligne2 = "", String ligne3 = "",
        String ligne4 = "") {
147   lcd.clear();
148   lcd.setCursor(0, 0); lcd.print(ligne1);
149   lcd.setCursor(0, 1); lcd.print(ligne2);
150   lcd.setCursor(0, 2); lcd.print(ligne3);
151   lcd.setCursor(0, 3); lcd.print(ligne4);
152   delay(2000);
153 }
154
155 // === Boucle principale ===
156 void loop() {
157   int potSite = analogRead(PIN_POT);
158   int potAzimut = analogRead(PIN_POT1);
159
160   float targetSite = map(potSite, 0, 4095, 0, MAX_SITE_ANGLE);
161   float targetAzimut = map(potAzimut, 0, 4095, 0, MAX_AZIMUT_ANGLE);
162
163   // Si les potentiom tres sont    0, passage en mode Bluetooth
164   if (potSite == 0 && potAzimut == 0) {
165     afficherLCD("MODE BLUETOOTH ACTIF", "CONNECTEZ VOUS...", "EN ATTENTE DE
          VALEUR", "ANGLE DE SITE");
166     while (!SerialBT.hasClient()) {
167       SerialBT.println("EN ATTENTE DE CONNEXION BLUETOOTH");
168       delay(1000);
169     }
170     SerialBT.println("CONNECTE! BIENVENUE");
171     SerialBT.println("VEUILLEZ ENTRER L'ANGLE CIBLE EN SITE(0,75)");
172     targetSite = lireBluetoothAngle("Entrez l'angle SITE (0-75)", 0,
          MAX_SITE_ANGLE);
173   }
174
175   currentAngle = lire_site();
176
177   // === Correction site ===
178   float angleError = targetSite - currentAngle;
179   while (abs(angleError) > ANGLE_MARGIN_SITE) {
180     if (angleError < 0) {
181       commande -= min(abs(angleError), PAS_CORRECTION);
182       digitalWrite(PIN_LEDR, HIGH);
183     } else {
184       commande += min(PAS_CORRECTION, angleError);
185       digitalWrite(PIN_LEDO, HIGH);
186     }
187     digitalWrite(PIN_LEDV, LOW);
188     commande = constrain(commande, 0, 180);
```

```
    myServo.write(commande);
    delay(200);

    currentAngle = lire_site();
    potSite = analogRead(PIN_POT);
    if (potSite != 0) targetSite = map(potSite, 0, 4095, 0, MAX_SITE_ANGLE);
    angleError = targetSite - currentAngle;

    afficherLCD("Angle Actuel: " + String(currentAngle, 1) + (char)223,
                "Angle Cible: " + String(targetSite, 1) + (char)223,
                "CORRECTION DE SITE", " EN COURS");
  }
  digitalWrite(PIN_LEDV, HIGH);
  digitalWrite(PIN_LEDO, LOW);
  digitalWrite(PIN_LEDR, LOW);
  afficherLCD(" CALAGE SITE OK");
  SerialBT.println("CALAGE SITE OK");

  // === Saisie angle Azimut via Bluetooth ===
  if (potSite == 0 && potAzimut == 0) {
    afficherLCD("SAISIE D'AZIMUT");
    targetAzimut = lireBluetoothAngle("Entrez l'angle AZIMUT (0-60)", 0,
        MAX_AZIMUT_ANGLE);
  }
  angleZ = lire_azimut();

  float angleError1 = targetAzimut - angleZ;
  while (abs(angleError1) > ANGLE_MARGIN_AZIMUT) {
    int sens = (angleError1 > 0) ? 1 : -1;
    digitalWrite(PIN_LEDO, sens == 1);
    digitalWrite(PIN_LEDR, sens != 1);
    digitalWrite(PIN_LEDV, LOW);

    int errorStep = map(abs(angleError1), 0, 360, 0, MAX_STEPPER_STEP);
    int pas = (errorStep > MIN_STEPPER_STEP) ? min(204, errorStep) :
        MIN_STEPPER_STEP;
    myStepper.step(sens * pas);
    delay(200);

    angleZ = lire_azimut();
    potAzimut = analogRead(PIN_POT1);
    if (potAzimut != 0) targetAzimut = map(potAzimut, 0, 4095, 0,
        MAX_AZIMUT_ANGLE);
    angleError1 = targetAzimut - angleZ;

    afficherLCD("Azimut actuel: " + String(angleZ, 1) + (char)223,
                "Azimut cible : " + String(targetAzimut, 1) + (char)223,
                "CORRECTION D'AZIMUT", " EN COURS");
  }
```

```
235   digitalWrite(PIN_LEDV, HIGH);
236   digitalWrite(PIN_LEDO, LOW);
237   digitalWrite(PIN_LEDR, LOW);
238   afficherLCD(" CALAGE AZIMUT OK");
239   SerialBT.println("CALAGE AZIMUT OK");
240
241   // === Affichage final ===
242   delay(500);
243   afficherLCD("Site Actuel : " + String(currentAngle, 1) + (char)223,
244               "Site Cible : " + String(targetSite, 1) + (char)223,
245               "Azimut Actuel: " + String(angleZ, 1) + (char)223,
246               "Azimut Cible : " + String(targetAzimut, 1) + (char)223);
247
248   delay(3000);
249   afficherLCD("Ecart Site : " + String(angleError, 1),
250               "Ecart Azimut: " + String(angleError1, 1),
251               "AUCUNE ERREUR", " DETECTEE");
252   SerialBT.println("CALAGE REUSSIE.");
253   delay(6000);
254 }
```

### 6.2.1. Explication des mots-clés du code

Afin de faciliter la compréhension du programme implémenté, les mots-clés essentiels employés dans le code, ainsi que leur rôle spécifique, sont présentés dans le tableau ci-après. Cette synthèse permet de mieux cerner la structure et la logique du code développé.

| Mot-clé / Élément | Description |
|---|---|
| `#include` | Directive pour inclure des bibliothèques nécessaires (Servo, MPU6050, Bluetooth, LCD, Stepper) |
| `define` | Déclare des constantes (pins, angles max, RPM, etc.) |
| `LiquidCrystal_I2C lcd(...)` | Objet pour contrôler un écran LCD I2C 20x4 |
| `Stepper myStepper(...)` | Objet pour contrôler un moteur pas à pas (ULN2003) |
| `BluetoothSerial SerialBT` | Permet la communication Bluetooth avec un smartphone |
| `Servo myServo` | Objet pour piloter le servomoteur (axe site) |
| `MPU6050 mpu` | Objet pour accéder aux données du capteur MPU6050 (accéléromètre + gyroscope) |
| `setup()` | Fonction d'initialisation exécutée une seule fois au démarrage |
| `loop()` | Fonction principale qui s'exécute en boucle |
| `analogRead()` | Lit la valeur d'un potentiomètre (0 à 4095 pour ESP32) |
| `map(...)` | Convertit une valeur d'un intervalle vers un autre (ex : potentiomètre → angle) |
| `pinMode(...)` | Configure une broche en entrée ou sortie |
| `digitalWrite(...)` | Active ou désactive une LED (rouge, orange, verte) pour afficher l'état |
| `readStringUntil('\n')` | Lit une chaîne reçue via Bluetooth jusqu'au saut de ligne |
| `constrain(...)` | Limite une valeur à un intervalle défini |
| `atan2(...)` | Fonction mathématique pour calculer l'angle à partir de l'accélération |
| `afficherLCD(...)` | Fonction personnalisée qui affiche des messages sur les 4 lignes du LCD |
| `delay(...)` | Pause en millisecondes entre les opérations |
| `while(...)` | Boucle de correction exécutée tant que l'erreur est supérieure à la marge |
| `min(...)` | Prend la plus petite valeur entre deux valeurs données |
| `SerialBT.println(...)` | Envoie un message au téléphone via Bluetooth |
| `angleZ, currentAngle` | Angles mesurés par le capteur (Z pour azimut, angle global pour site) |
| `PAS_CORRECTION` | Valeur de base de rotation à appliquer pour corriger un angle |
| `MAX_STEPPER_STEP` | Nombre maximum de pas d'un tour complet du moteur pas à pas |
| `alpha` | Constante utilisée pour le filtrage complémentaire des données MPU6050 |
| `lcd.setCursor(...)` | Place le curseur à une position donnée (colonne, ligne) sur le LCD |

TABLE 6.2 – Résumé des mots-clés du code et leurs fonctions

## 6.3. Evaluation des coûts

L'évaluation des coûts a été effectuée sur la base des composants effectivement utilisés pour la réalisation du prototype. Les prix indiqués ci-dessous sont estimatifs, basés sur les tarifs constatés au moment de l'achat ou sur les factures reçues.

| Composant | Quantité | Prix unitaire (FCFA) |
|---|---|---|
| carte ESP32 | 1 | 7 000 |
| MPU6050 (commandé depuis Madagascar) | 1 | 5 500 |
| Servomoteur SG90 | 4 (dont 3 défectueux) | 2 500 |
| Moteur pas à pas 28BYJ-48 + ULN2003 | 1 | 2 500 |
| Écran LCD 20x4 I2C | 1 | 4 500 |
| Potentiomètres (10kΩ) | 2 | 500 |
| Plaques d'essai et câbles Dupont | 2 lot | 2 500 |
| LEDs (rouge, orange, verte) | 3 | 100 |
| Divers (déplacement, livraison, caisse et soudures...) | - | 25 000 |
| **Total estimé** | | **60 800 FCFA** |

TABLE 6.3 – Tableau estimatif des coûts du prototype

## 6.4. Difficultés rencontrées

Malgré la réussite globale du projet, plusieurs obstacles techniques et logistiques ont été rencontrés tout au long de sa mise en œuvre.
L'une des principales difficultés rencontrées au cours du projet a été l'approvisionnement en composants électroniques à Niamey. Le marché local est peu fourni et il est souvent difficile d'entrer en contact avec des fournisseurs fiables. La plupart des composants disponibles étaient soit relativement coûteux, soit de qualité inférieure, ce qui a freiné l'avancement du projet à plusieurs reprises.

Certains modules essentiels, comme le capteur MPU6050, n'étaient tout simplement pas disponibles sur place. Nous avons dû passer par un contact personnel basé à Madagascar pour en obtenir un, ce qui a occasionné des délais. Par ailleurs, plusieurs composants ont dû être changés plusieurs fois au cours des tests, en raison de dysfonctionnements récurrents. C'est notamment le cas du servomoteur, que nous avons remplacé à trois ou quatre reprises avant de stabiliser le système. Ces contraintes ont nécessité une adaptation constante et ont renforcé notre capacité à diagnostiquer rapidement les pannes et à réagir efficacement face aux imprévus.

## 6.5. Perspectives d'améliorations

Le système conçu satisfait pleinement aux exigences de calage automatique de l'angle d'inclinaison d'un plan. Néanmoins, dans une perspective d'optimisation continue et d'enrichissement

fonctionnel, quelques axes d'amélioration peuvent être envisagés :

- **Alimentation autonome** : utiliser une batterie rechargeable (par exemple Li-Ion) avec un circuit de gestion de charge pour rendre le montage mobile et indépendant du secteur ;
- **Remplacement du servomoteur par un vérin** : pour des applications réelles ou à plus grande échelle, le servomoteur pourrait être remplacé par un vérin électrique, capable de fournir plus de couple et de précision pour le contrôle de l'inclinaison en site ;
- **Amélioration de la précision** : utiliser des capteurs plus stables comme le `BNO055`, et mettre en œuvre un filtrage plus performant (ex. filtre de Kalman) pour une meilleure estimation angulaire.

EAMAC

Chapitre 7

# Applications

Le système mis en œuvre présente un fort potentiel d'applications concrètes dans des secteurs variés, tels que l'aéronautique et les énergies renouvelables. Il peut notamment être mobilisé pour assurer le calage automatique du PAPI (dispositif visuel d'aide à l'atterrissage), des feux d'approche haute intensité et des feux flash ainsi que pour optimiser l'orientation de panneaux photovoltaïques en fonction de la trajectoire solaire.

## 7.1. Le PAPI

### 7.1.1. Définition et fonctionnement

Le PAPI (Precision Approach Path Indicator) est un système visuel d'aide à l'atterrissage installé à proximité des pistes d'aérodrome. Il se compose généralement de quatre unités lumineuses alignées perpendiculairement à l'axe de piste. Ces lampes projettent des faisceaux lumineux rouges et blancs visibles par le pilote à l'approche. En fonction de l'angle d'approche de l'avion, le pilote perçoit une combinaison de couleurs : deux lampes blanches et deux rouges indiquent une pente correcte, tandis que quatre rouges signalent une approche trop basse et quatre blanches une approche trop haute.

Le principe de fonctionnement repose sur la perception angulaire des faisceaux lumineux. Chaque unité lumineuse est réglée à un angle précis par rapport à la pente d'approche nominale. Grâce à des filtres et lentilles, la lumière perçue change de couleur selon l'angle sous lequel elle est observée. Ce système ne nécessite aucun équipement embarqué, ce qui le rend universellement utilisable, de jour comme de nuit, pour guider les pilotes vers une trajectoire d'atterrissage sécurisée.




FIGURE 7.1 – Le PAPI

### 7.1.2. Apport du système au PAPI

Le PAPI repose sur l'orientation rigoureuse de blocs lumineux selon un angle d'inclinaison précisément défini, généralement compris entre 2° et 4°. Actuellement, ce réglage s'effectue manuellement à l'aide d'instruments optiques, une méthode à la fois chronophage et sujette à des imprécisions. Le système proposé dans le cadre de ce projet permettrait d'automatiser ce processus à l'aide d'un servomoteur ou d'un vérin asservi en temps réel par un capteur d'inclinaison de type MPU6050. L'ensemble serait complété par un affichage local sur écran LCD et une interface de configuration à distance via liaison Bluetooth ou WIFI. Une telle solution garantirait un calage plus précis, une mise en œuvre accélérée et une utilisation possible tant lors de l'installation initiale que lors des opérations de maintenance périodique.

## 7.2. Les panneaux photovoltaïques

### 7.2.1. Définition et fonctionnement

Les panneaux photovoltaïques sont des dispositifs conçus pour convertir directement l'énergie lumineuse du soleil en électricité. Cette conversion repose sur le principe de l'effet photovoltaïque, découvert par Edmond Becquerel en 1839. Chaque panneau est constitué de cellules photovoltaïques, généralement en silicium, qui captent les photons de la lumière solaire. Lorsque ces photons frappent la surface des cellules, ils libèrent des électrons, générant ainsi un courant électrique continu. Les panneaux peuvent être assemblés en séries et en parallèles pour produire la tension et la puissance souhaitées selon les besoins de l'installation.

Le fonctionnement optimal des panneaux photovoltaïques dépend fortement de leur orientation par rapport au soleil. Une mauvaise inclinaison ou un mauvais alignement réduit considérablement leur rendement.



FIGURE 7.2 – Les panneaux photovoltaïques

### 7.2.2. Apport du système au power tracking

Le système développé, capable de corriger automatiquement les angles en site et en azimut, constitue une base fonctionnelle pour la réalisation d'un suiveur solaire à deux axes. Grâce à l'utilisation d'un capteur d'orientation (MPU6050) et de moteurs (servomoteur et moteur pas à pas), l'orientation du panneau peut être ajustée dynamiquement selon des données programmées ou transmises à distance via Bluetooth ou WIFI. Il serait également possible d'y intégrer un algorithme de trajectoire solaire ou des capteurs de lumière (LDR) pour rendre le système entièrement autonome. Ce type de solution, simple et économique, est particulièrement adapté aux centrales solaires décentralisées ou aux sites isolés où le rendement énergétique doit être optimisé avec des moyens limités.

# Conclusion

En définitive, ce projet a abouti à la conception et à la réalisation d'un dispositif fonctionnel de calage automatique de l'angle d'inclinaison d'un plan, reposant sur l'intégration d'un microcontrôleur ESP32 et d'un capteur inertiel MPU6050. Ce système innovant permet la détection en temps réel de l'orientation du plan, suivie d'un ajustement autonome de sa position au moyen d'un servomoteur et d'un moteur pas à pas.

La validation du dispositif, menée dans un premier temps par simulation sous Proteus, puis consolidée par des expérimentations pratiques, a démontré la faisabilité technique ainsi que l'efficacité du système, en dépit de certaines limitations matérielles. Ce travail ouvre ainsi des perspectives d'application concrètes, notamment dans les domaines du balisage lumineux d'approche (PAPI et feux HI ) ou de l'optimisation de l'orientation des panneaux photovoltaïques. Il laisse entrevoir des axes d'amélioration prometteurs, tant en termes de précision que de robustesse.

EAMAC

# Bibliographie


[1] https://zestedesavoir.com/membres/voir/Eskimon/, *Arduino : Premiers pas en informatique embarquée*, PDF, consulté en 2025.

[2] Astalaseven, Eskimon et Olyte, *Arduino : Pour bien commencer en électronique et en programmation*, PDF, consulté en 2025.

[3] EAMAC, *Cours EAM-FR5.4-ISEI-AFM2-YY-B - Automates figés et microcontrôleurs*, Document pédagogique, 2025.

[4] Erik Bartmann, *Le grand livre d'Arduino*, PDFDrive, consulté en 2025.

[5] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, 2013. Disponible en ligne : https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf

[6] Espressif Systems, *ESP32-WROOM-32 Datasheet*, 2023. Disponible en ligne : https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

[7] Tower Pro, *SG90 Micro Servo Motor Datasheet*, consulté en 2025. Disponible en ligne : https://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sg90_datasheet.pdf

[8] ETC, *28BYJ-48 Stepper Motor Datasheet*, consulté en 2025. Disponible en ligne : https://datasheetspdf.com/pdf/746291/ETC/28BYJ-48/1

[9] Texas Instruments, *ULN2003A Datasheet*, 2002. Disponible en ligne : https://www.ti.com/lit/ds/symlink/uln2003a.pdf

[10] Hitachi, *HD44780 LCD Controller Datasheet*, 2001. Disponible en ligne : https://www.sparkfun.com/datasheets/LCD/HD44780.pdf

[11] Bourns Inc., *3386 Series – 10k Potentiometer Datasheet*, 2017. Disponible en ligne : https://www.bourns.com/docs/Product-Datasheets/3386.pdf

[12] Borne, P., & Sita, J., *Capteurs et instrumentation : Mesures physiques et capteurs intelligents*, Éditions Technip, 2011.

[13] IUT GEII, *Cours Capteurs & Instrumentation – DUT Génie Électrique*, consulté via cours-de-elec.fr, geii.fr, www.arduino.cc, 2025.

[14] Maxicours.com, *Acquérir l'information — Les différents capteurs*, Disponible en ligne : https://www.maxicours.com/se/cours/acquerir-l-information---les-differents-capteurs/, consulté en 2025.